

Waste & Trash Detection

How to think about it, how to build it, and everything to study, using three non-YOLO detectors: Faster R-CNN, RetinaNet and SSD300.

Prepared for Sreyanshu and team · 24 September 2026 · Evaluation on 29–30 September

At a glance

Goal	Draw a box around every waste item in a photo, name its class (8 classes), and compare three detection models.
Data	163 labelled street-trash photos (Kaggle, Pascal VOC / COCO / YOLO labels). Classes: Bottle, Bottle Cap, Can, Cardboard, Cigarette Box, Cigarette Butt, Cup, Plastic.
Models	Faster R-CNN (two-stage), RetinaNet (one-stage with focal loss), SSD300 (one-stage, multi-scale, fast). All come pretrained on COCO from torchvision. No YOLO, no Ultralytics.
Tools	Google Colab (free T4 GPU), PyTorch + torchvision, torchmetrics. No Kaggle login.
You deliver	A notebook that runs end to end, three trained models, a results table and figures, a report, and a demo script on every laptop.
Effort	About 20–25 hours per person over five days. Training needs only about an hour of GPU time in total.
Companion file	<code>trash_detection_colab.ipynb</code> is the complete, tested notebook. Chapter 10 walks through it step by step.

How to use this guide

Read chapters 1 and 2 today: the plan and the thinking behind every decision. Chapters 3–9 are the study notes. Each ends with a **Be ready to explain** box listing what an examiner can ask. Chapter 10 walks through the notebook step by step. Chapters 11–13 cover the report, the viva and fixes for common problems. The appendices have a one-page formula sheet, a glossary and references.

Before this guide was written, the notebook was run end to end on a small stand-in dataset with the same file formats and quirks: data loading, orientation fixes, splitting, training all three models, every metric and chart, and the laptop script. The hand-written matching code agrees with the official COCO-style mAP to within 0.001. Two things couldn't be run in that test environment: the Kaggle download, and a GPU run starting from the downloaded COCO weights. Both use the standard torchvision and Colab routes. If the download fails, the notebook explains a two-minute browser workaround, and chapter 13 covers the likely GPU hiccups.

Contents

1 The 5-day plan	4
1.1 Roles	4
1.2 Definition of done (Monday evening)	4
1.3 Working as a team	5
2 The thought process	6
2.1 Read the brief like an engineer	6
2.2 Seven decisions that shape the project	6
2.3 The experiment design	8
2.4 The pipeline	8
2.5 What to expect: hypotheses, not results	8
2.6 Ten pitfalls, and how this project avoids them	9
3 Deep-learning refresher	10
3.1 From pixels to feature maps	10
3.2 Backbone, neck and head	10
3.3 Transfer learning and fine-tuning	10
3.4 One training step, and the knobs that control it	10
3.5 Overfitting, and four defences	11
3.6 Train, validation and test: three different jobs	11
4 How object detectors work	12
4.1 The task	12
4.2 Bounding boxes and their formats	12
4.3 Intersection over Union (IoU)	12
4.4 Anchors (default boxes)	13
4.5 Box regression: predict corrections, not coordinates	14
4.6 The imbalance problem, the thread through all three models	14
4.7 Non-maximum suppression (NMS)	14
4.8 Feature Pyramid Network (FPN): small and large objects	14
4.9 The families of detectors	15
5 The three models	16
5.1 Faster R-CNN (two-stage)	16
5.2 RetinaNet (one-stage, focal loss)	17
5.3 SSD300 (one-stage, multi-scale, fast)	19
5.4 Side by side	19
6 Data: formats, cleaning and augmentation	21
6.1 What's in the download, and what we use	21
6.2 The three label formats, in the files	21
6.3 Cleaning the labels (notebook Step 2)	21
6.4 Exploring the data (Step 3)	22

6.5 Splitting (Step 4)	22
6.6 Augmentation for detection (Step 5)	22
6.7 Class imbalance	23
7 Training	24
7.1 The recipe	24
7.2 One iteration, in code order	24
7.3 Reading the training curves	25
7.4 Time and memory on a Colab T4	25
8 Evaluation metrics	26
8.1 Matching predictions to objects	26
8.2 Precision, recall and F1	26
8.3 The confidence threshold	26
8.4 Precision-recall curve and Average Precision (AP)	26
8.5 The mAP family	27
8.6 The confusion matrix for detection	28
8.7 Speed and size	28
8.8 How much to trust the numbers	28
9 Error analysis	29
9.1 The error types	29
9.2 From error type to cause to remedy	30
9.3 Recall by object size	30
9.4 Choosing example images	30
9.5 Writing an observation	30
10 Build guide, step by step	31
Step 0 — Settings, Drive and folders	31
Step 1 — Download the dataset	31
Step 2 — Read the labels and make a clean copy	32
Step 3 — Explore and verify	33
Step 4 — Split 70 / 15 / 15	33
Step 5 — Dataset, augmentation and the three models	33
Step 6 — Train	35
Step 7 — Results on the test set	36
Step 8 — Error analysis	37
Step 9 — Unseen photos, and every laptop	38
11 Writing the report	39
11.1 Structure	39
11.2 Tables to fill in	39
11.3 Writing the conclusion	40
11.4 Limitations and future work	40
11.5 Style tips	40
12 Viva preparation	41
12.1 The problem and the data	41
12.2 The models	41
12.3 Training	42
12.4 Metrics	43
12.5 Results and analysis	44
12.6 Practical	44
13 Troubleshooting	45
13.1 The two-minute sanity check	45
Appendix A • Formula sheet	46
Appendix B • Glossary	47
Appendix C • References	48

1 The 5-day plan

Today is Thursday 24 September. The evaluation is on 29–30 September, which leaves five working days, Thursday to Monday. Tuesday is a buffer only if your slot is on the 30th. The plan assumes 4–5 focused hours per person per day. Training is quick, so most of the time goes into understanding the models, analysing results and writing.

Day	Study (everyone)	Build	Done when...
Thu 24	Ch. 2–4: the thought process, refresher, how detectors work	Data lead runs Steps 0–4: download, clean copy, box check, split. Shares <code>split.json</code> . Everyone else runs Steps 0–3 in their own Colab to learn the pipeline. Start Faster R-CNN training before you stop (about 20 min).	8 classes printed; <code>label_check.png</code> shows boxes sitting on objects; split shared
Fri 25	Ch. 5–7: the three models, data, training	Each model owner trains their model with the shared split. Look at the training curves together.	3 checkpoints in Drive; loss falls; validation mAP rises then levels off
Sat 26	Ch. 8–9: metrics, error analysis	Evaluation owner runs Steps 7–8 with all three checkpoints. Everyone photographs 5–10 real waste scenes for Step 9. Teach-back: each owner explains their model for 15 minutes.	<code>summary.csv</code> , <code>per_class.csv</code> , <code>errors.csv</code> and figures saved; numbers make sense
Sun 27	Ch. 11: the report	Write the report from the template; split the sections; agree the observations together.	Complete draft with every table and figure
Mon 28	Ch. 12: viva questions	Finalise the report. Copy the whole project to every laptop and run <code>predict.py</code> on each. Mock viva in pairs.	Every box in the checklist below is ticked
Tue 29	Buffer, only if you present on the 30th: fix whatever the mock viva exposed.		

1.1 Roles

For a team of four. With three, merge roles 1 and 4. Everyone still has to understand every part; the teach-back and the mock viva make sure of that.

Role	Owns
1 • Data & pipeline lead	Steps 1–5, <code>split.json</code> , the Dataset section of the report
2 • Faster R-CNN owner	Trains it; explains two-stage detection and the RPN
3 • RetinaNet owner	Trains it; explains focal loss and class imbalance
4 • SSD300 & evaluation owner	Trains SSD300; runs Steps 7–8; owns the metrics and error-analysis sections

1.2 Definition of done (Monday evening)

- ☐ The notebook runs top to bottom in Colab without errors.
- ☐ `split.json`, three checkpoints and the `results/` folder are saved in Drive.
- ☐ Results table for all three models: mAP@0.5:0.95, mAP@0.5, precision, recall, F1, speed, size.
- ☐ Per-class table, plus at least four example images showing correct, missed and false detections.
- ☐ Error breakdown, confusion matrices and recall by object size.
- ☐ Your own unseen photos tested, with written observations.
- ☐ Report finished (chapter 11) and exported to PDF.
- ☐ Every laptop has the project folder and the report, and `predict.py` has been tested on it.
- ☐ Every member can answer the questions in chapter 12 without notes.

1.3 Working as a team

- **Who trains what.** The simplest option is one person training all three models back to back, about an hour of GPU time. Better for learning: each owner trains their own model in their own Colab. The free GPU quota is per account, so this also spreads the load.
- **Share files, not screens.** Everyone keeps their own `MyDrive/trash_detection/` folder. The data lead sends `split.json` to everyone *before* they run Step 4. Each owner uploads their `checkpoints/<model>.pt` and `results/history_<model>.json` to the evaluation owner's folders, who then runs Steps 7–9 for all three.
- **One master notebook.** If anyone changes code, they say what and why in the team chat. A silent change to a setting breaks the fair comparison.
- **Keep a decision log.** One shared note with the split seed, each model's best epoch and threshold, surprises and observations. Half your report's Methods and Discussion sections will come from it.
- **Daily 10-minute check-in.** What's done, what's next, what's blocking. Sort out blockers the same day.

2 The thought process

A good project isn't a pile of code that happens to run. It's a chain of decisions you can defend. This chapter is that chain: what the brief really asks for, the decisions that follow from it, and how to set up a comparison that an examiner will trust.

2.1 Read the brief like an engineer

Translate every line of the case study into something you will build or measure. If a requirement has no deliverable next to it, you've missed something.

The brief says	Technically, that means	You will produce
Detect waste objects in images	Object detection: a box around every item, not one label per photo	Three trained detectors
Identify the category of each item	Each box gets one of 8 classes plus a confidence score	Class and score for every box
Objects may overlap	Touching boxes; duplicate removal (NMS) must not delete real neighbours	An error type for overlaps, with examples
Different shapes, sizes and backgrounds	Scale variation; tiny items (butts, caps) are the hard cases	Recall by object size; per-class results
Use bounding-box annotations	Pascal VOC boxes, checked by drawing them	A label-check figure
Test on unseen images	A held-out test set, never used for training or tuning	Test-set results plus your own new photos
Compare at least three models	Three different designs under one identical protocol	A comparison table and charts
Precision, recall and mAP	Standard COCO/VOC metrics at IoU 0.5 and 0.5:0.95	<code>summary.csv</code> , <code>per_class.csv</code>
Correct, missed and false detections	TP, FN and FP counts, each with a reason	<code>errors.csv</code> , confusion matrices, example images
Clear final report	Results → observations → conclusion	The report (chapter 11)

2.2 Seven decisions that shape the project

D1

Use detectors pretrained on COCO, then fine-tune them (transfer learning)

About 114 training photos is far too few to learn what edges, textures and objects look like from scratch. Models trained on COCO (118,000 images, 80 everyday object types) already know all of that. Two COCO classes are even *bottle* and *cup*. We replace only the final classification layer (91 COCO outputs → 9 outputs: our 8 classes plus background) and fine-tune the rest on our photos.

D2 Compare three genuinely different designs

Faster R-CNN first proposes regions, then classifies each one (two stages). **RetinaNet** classifies every anchor directly and uses focal loss to cope with the flood of background (one stage). **SSD300** predicts from six feature maps of a small 300×300 image (one stage, built for speed). They are three answers to the same two questions: *how much accuracy is worth how much speed*, and *how do you stop the background from swamping the learning*. All three come from torchvision, so they share one data pipeline and one training loop. Any difference in the results comes from the models, not the code.

Why not YOLO: it's not allowed. **Why not DETR-style transformers:** they need very long training (the original DETR trained for 300–500 epochs on COCO) and a lot of data, which is a poor fit for 114 photos and five days. Reasonable alternatives if you ever need one: FCOS (anchor-free), Faster R-CNN with a MobileNet backbone, and SSDlite. All are in torchvision.

D3 Split the 163 original photos yourself, before any augmentation

The download also contains 4,923 augmented images. These are flips, colour changes and crops of the same 163 photos. If an augmented copy of a test photo lands in the training set, the model has effectively seen the test photo, and the test score is inflated. This is called **data leakage**. We split the originals 70 / 15 / 15 (114 / 24 / 25 images) and save the split to `split.json`, so no photo or copy of it can appear in two sets. We use 15% for testing rather than the dataset's 10% (16 images) to make the test estimate a little steadier.

D4 Augment on the fly

Instead of the fixed augmented images, the training pipeline makes a new random variation of each training photo every epoch: colour changes, zoom in or out, horizontal flip. The boxes move with the pixels. In 30 epochs this gives more variety than the pre-made set, and it only ever touches training images.

D5 Make the comparison fair: change only the model

Same images, split, augmentation, optimizer, learning rate, schedule, epochs, batch size, seed, post-processing and metric code for all three. Section 2.3 lists it all. If one model got more epochs or its own tuning, you couldn't say whether it won because of its design or because of the extra care.

D6 Choose on validation, measure on test once

Two choices depend on data: which epoch to keep, and which confidence threshold to use for precision and recall. Both are made on the **validation** set. The **test** set is used once, at the end. If you picked the threshold on the test set, the test score would be tuned to those 25 photos and would overstate how well the model works on new ones.

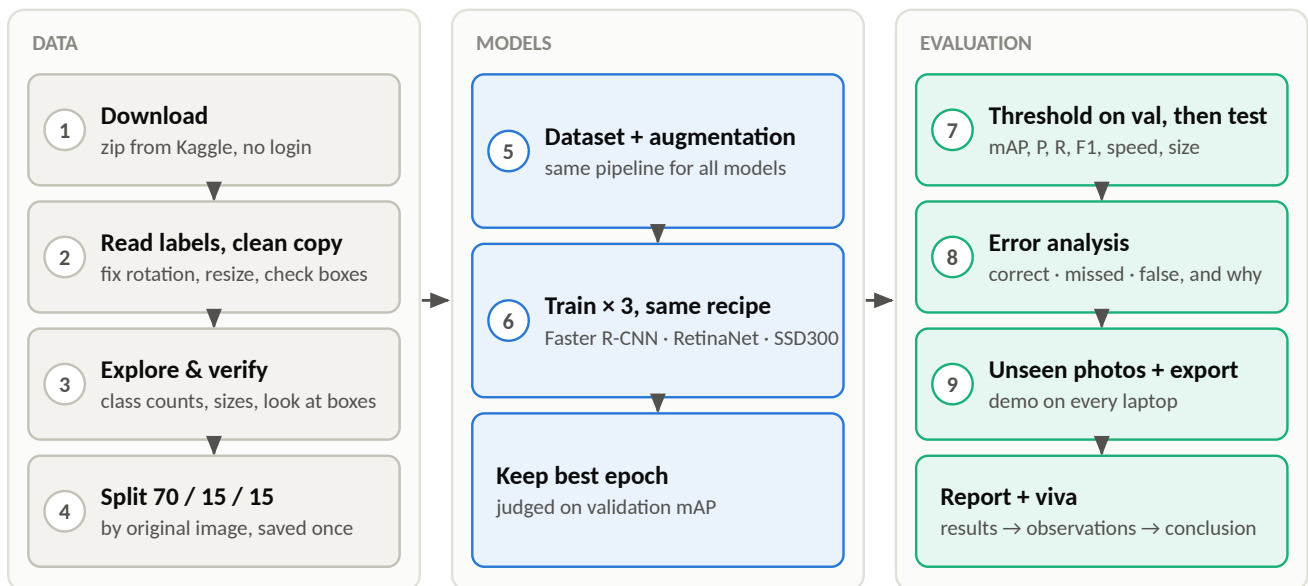
D7 Measure what a waste-sorting line would care about

Accuracy (mAP), how many items are found (recall), how many alarms are false (precision), speed per image and model size. A sorting line runs in real time, so a model that is slightly less accurate but five times faster can be the better recommendation.

2.3 The experiment design

Kept identical for all three models	Allowed to differ (part of each design)
- Images, split (<code>split.json</code>) and label cleaning - Augmentation pipeline - COCO-pretrained starting weights - AdamW, learning rate 1e-4, weight decay 1e-4 1-epoch warm-up, then cosine decay; 30 epochs; batch 4 - Mixed precision, gradient clipping, seed 42 - Best epoch = highest validation mAP@0.5:0.95 - Post-processing: score ≥ 0.01, NMS IoU 0.5, at most 100 boxes per image - Threshold = best F1 on validation - Test set, metric code, GPU (Colab T4)	- Architecture: two-stage vs one-stage - Backbone: ResNet-50 + FPN vs VGG-16 - Input size: short side 800 px vs a fixed 300×300 (SSD300 is built for 300) - Anchor layout - Loss functions: cross-entropy + smooth L1, focal + GIoU, cross-entropy with hard negative mining These differences <i>are</i> the models, so they are what we compare. State them in the report so nobody thinks you gave one model an unfair advantage.

2.4 The pipeline



The project from download to report. The numbers match the notebook steps and chapter 10.

2.5 What to expect: hypotheses, not results

Write your expectations down *before* you see the numbers. Then test them. Your analysis becomes a story: hypothesis → evidence → verdict. Examiners like that much better than a list of numbers.

- **H1.** Faster R-CNN will be the most accurate, especially on small objects. It refines each proposal a second time, uses an FPN and sees the photo at 800 px.
- **H2.** SSD300 will be the fastest, but the weakest on small items such as cigarette butts and bottle caps, because it only sees a 300×300 version of the photo.
- **H3.** RetinaNet will come close to Faster R-CNN in accuracy (focal loss handles the background imbalance) and run somewhat faster.
- **H4.** Most confusions will be between look-alike classes: Cup vs Bottle vs Plastic, Cigarette Box vs Cardboard.

Small test set. 25 test photos means a single object can move a per-class score a lot. Don't declare a winner from a one- or two-point gap. Look for clear gaps and patterns that repeat across metrics and classes.

Signs of a bug, not a weak model: validation mAP still near 0 after several epochs; boxes that are consistently shifted or rotated in the example images; a class that is never predicted; a loss of `nan`. Chapter 13 covers each one.

2.6 Ten pitfalls, and how this project avoids them

#	Pitfall	How we avoid it
1	Boxes don't line up with the photos (phone rotation, EXIF)	Step 2 detects and fixes it; Step 3 lets you check by eye
2	Leakage through augmented copies	Split the originals; augment on the fly (D3, D4)
3	Tuning on the test set	Epoch and threshold chosen on validation (D6)
4	Class ids off by one	0 = background, labels 1-8, <code>num_classes = 9</code>
5	Wrong box format	VOC and torchvision both use xmin, ymin, xmax, ymax in pixels
6	Comparing loss values across models	Each model defines its loss differently: compare mAP, not loss
7	Precision and recall without a threshold	Always state the threshold next to P and R
8	Picking a winner from a tiny gap	Look at per-class results and consistency (2.5)
9	Colab disconnects and work is lost	Split, checkpoints and results are saved to Google Drive
10	Only one person understands the code	Teach-back on Saturday, mock viva on Monday

3 Deep-learning refresher

You know Python and the basics of machine learning. This chapter covers only the deep-learning ideas this project depends on, in the words you'll need in the viva.

3.1 From pixels to feature maps

A convolution layer slides small filters (typically 3×3) across the image, and each filter responds to one pattern. Stack many layers and the patterns build up: early layers react to edges and colours, middle layers to textures and parts (a bottle neck, a ridged can), deep layers to whole object-like shapes.

Along the way the grid shrinks. ResNet-50 reduces the resolution by 4, 8, 16 and then 32. An 800×1067 input becomes a 200×267 grid at stride 4 and a 25×34 grid at stride 32. Each cell of that grid is a **feature vector** of 256–2,048 numbers describing one patch of the photo. The whole grid is a **feature map**.

Why this matters here. A 20-pixel cigarette butt covers less than one cell of a stride-32 feature map. Small objects need high-resolution feature maps, which is exactly what the FPN in Faster R-CNN and RetinaNet provides (section 4.8).

3.2 Backbone, neck and head

Part	Job	In our three models
Backbone	Turn the image into feature maps	ResNet-50 (Faster R-CNN, RetinaNet); VGG-16 (SSD300)
Neck	Combine feature maps of different resolutions	Feature Pyramid Network (Faster R-CNN, RetinaNet). SSD300 uses its extra layers directly.
Head	Predict classes and boxes	RPN + box head (Faster R-CNN); class and box subnets (RetinaNet); one small conv per feature map (SSD300)

3.3 Transfer learning and fine-tuning

Our models went through two rounds of training before we got them. The backbone learned general visual features on ImageNet (1.2 million photos), then the whole detector learned to find 80 object types on COCO. We take that detector and:

1. **Replace the final classification layer.** COCO's layer outputs 91 scores (80 classes, unused ids and background). Ours outputs 9: background plus our 8 classes. This layer starts with random weights.
2. **Fine-tune.** Train on our photos with a small learning rate, so the pretrained layers adapt gently and don't forget what they know.

By default torchvision freezes the earliest backbone layers, which detect generic edges that don't need to change. Faster R-CNN and RetinaNet train the last 3 of the 5 ResNet stages; SSD300 trains the last 4 of the 5 VGG blocks.

3.4 One training step, and the knobs that control it

With 114 training images and a batch of 4, one **epoch** (one pass over every training image) is 29 **iterations**. Thirty epochs are about 870 weight updates. Each iteration runs the batch through the model, computes the loss, backpropagates the gradients and lets the optimizer update the weights.

Setting	Our value	What it does, and why
Optimizer: AdamW	lr 1e-4	Gives every weight its own step size, based on the recent size of its gradients. Weight decay is applied separately ("decoupled"). Robust with little tuning, which is why we can use the same setting for all three models.
Weight decay	1e-4	Shrinks weights slightly every step. A mild brake on overfitting.
Warm-up	1 epoch	The new classification layer starts random. Full-size steps straight away would push its large random gradients into the pretrained layers, so the learning rate ramps up from almost 0 during the first epoch.
Cosine decay	to 0	The learning rate then falls smoothly along half a cosine curve: big steps early to learn, tiny steps at the end to settle.
Gradient clipping	norm 10	Caps the rare huge gradient, for example from an odd augmented image, so one bad batch can't wreck the weights.
Mixed precision (AMP)	on for GPU	Does most of the maths in 16-bit floats, which the T4's tensor cores run roughly twice as fast. A gradient scaler stops tiny gradients rounding to zero.
Seed	42	Makes runs repeatable, apart from small GPU non-determinism.

3.5 Overfitting, and four defences

With 114 images, a 40-million-parameter network can simply memorise the training photos. The sign is that **training loss keeps falling while validation mAP stalls or drops**. We defend against it four ways:

1. **Pretrained weights:** we start from general knowledge rather than from nothing.
2. **Augmentation:** the model never sees exactly the same training image twice.
3. **Weight decay:** gently discourages extreme weights.
4. **Keeping the best validation epoch** (early stopping): if later epochs overfit, we simply don't use them.

3.6 Train, validation and test: three different jobs

Set	Images	Job	Exam analogy
Train	114 (70%)	Learn the weights	Practice problems with worked answers
Validation	24 (15%)	Make decisions: best epoch, confidence threshold	A mock exam used to plan revision
Test	25 (15%)	One honest final estimate, touched once	The real exam

Be ready to explain

- What a feature map is, and why a stride-32 map struggles with a 20-px object
- What the backbone, neck and head each do in our models
- Transfer learning: what we keep, what we replace (91 → 9 outputs) and why
- Why we warm up the learning rate, and what AdamW does
- How you would spot overfitting in your training curves, and the four defences
- Why we need a validation set as well as a test set

4 How object detectors work

All three models solve the same problem with the same building blocks: boxes, IoU, anchors, box regression, a way to handle the flood of background, and non-maximum suppression. Learn these once and the three architectures in chapter 5 become variations on a theme.

4.1 The task

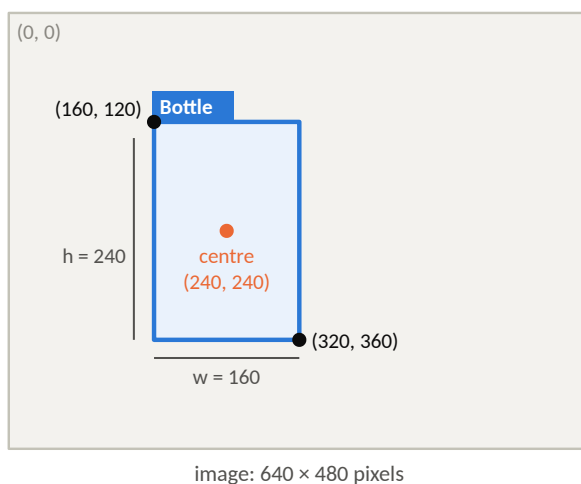
Classification gives one label per image. **Detection** finds every object: an unknown number of (box, class, confidence) triples. **Segmentation** labels every pixel, which we don't need. In evaluation mode, a torchvision detector returns one dictionary per image:

```
{  
  "boxes": tensor([[112.3, 40.8, 198.0, 260.1],  
                  [530.2, 610.5, 571.9, 640.0]]),  
  "labels": tensor([1, 6]),  
  "scores": tensor([0.97, 0.64])  
}
```

xmin, ymin, xmax, ymax in pixels
1 = Bottle, 6 = Cigarette Butt (0 = background)
confidence of each box

4.2 Bounding boxes and their formats

The same box can be written three ways, and the dataset ships all three. Mixing them up is one of the most common detection bugs.



Pascal VOC (.xml, one file per image)

xmin ymin xmax ymax, in pixels

160 120 320 360

COCO (.json, one file for all images)

x y width height, in pixels

160 120 160 240

YOLO (.txt, one file per image)

class cx cy w h, as fractions 0-1

0 0.375 0.500 0.250 0.500

One box in the three formats. torchvision uses the Pascal VOC layout: xmin, ymin, xmax, ymax in pixels.

VOC → COCO

$x = \text{xmin}$, $y = \text{ymin}$, $w = \text{xmax} - \text{xmin}$, $h = \text{ymax} - \text{ymin}$

VOC → YOLO

$cx = (\text{xmin} + \text{xmax}) / 2 / W$, $cy = (\text{ymin} + \text{ymax}) / 2 / H$, $w = (\text{xmax} - \text{xmin}) / W$, $h = (\text{ymax} - \text{ymin}) / H$

YOLO → VOC

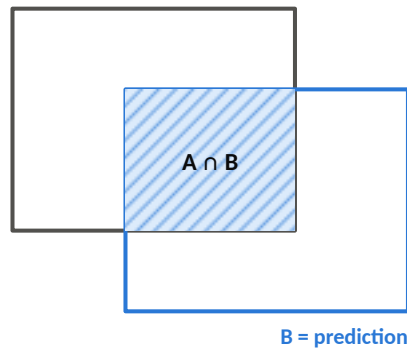
$\text{xmin} = (cx - w/2) \cdot W$, $\text{ymin} = (cy - h/2) \cdot H$, $\text{xmax} = (cx + w/2) \cdot W$, $\text{ymax} = (cy + h/2) \cdot H$

4.3 Intersection over Union (IoU)

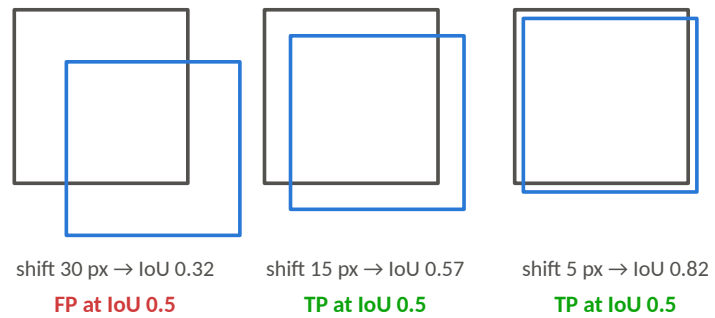
IoU measures how well two boxes agree: the area they share divided by the area they cover together. It runs from 0 (no overlap) to 1 (identical).

$$\text{IoU} = \frac{\text{area}(A \cap B)}{\text{area}(A \cup B)} = \frac{\text{overlap}}{\text{area}(A) + \text{area}(B) - \text{overlap}}$$

A = ground truth



The same 100 × 100 px object; the prediction is shifted in x and y



Left: the overlap (hatched) and the union. Right: how quickly IoU falls as a prediction drifts. The same 5-pixel error that barely matters for a large object pushes a 20 × 20-px object down to IoU 0.39, below the 0.5 cut-off.

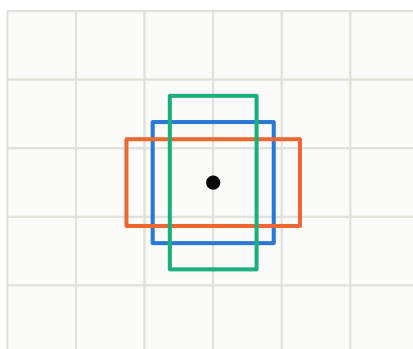
Worked example. Ground truth (100, 100, 300, 300) and prediction (150, 150, 350, 350). The overlap is $150 \times 150 = 22,500 \text{ px}^2$. The union is $40,000 + 40,000 - 22,500 = 57,500 \text{ px}^2$. $\text{IoU} = 22,500 / 57,500 = 0.39$. At the usual 0.5 threshold this prediction is a false positive, even though it clearly "found" the object.

IoU is used in three places: to **assign anchors** during training (4.4), to **remove duplicates** (NMS, 4.7), and to **score predictions** during evaluation (chapter 8).

4.4 Anchors (default boxes)

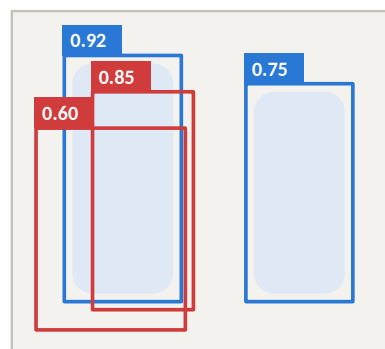
A detector can't just "output some boxes". It needs a fixed structure to predict into. **Anchors** provide it: reference boxes of a few sizes and shapes, placed at every position of a feature map. For every anchor the network answers three questions. Is there an object here? Which class is it? How should this anchor be shifted and stretched to fit it?

Anchors: 3 shapes at every grid position



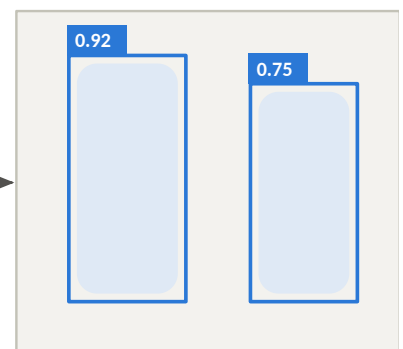
aspect ratios 1:1, 2:1, 1:2 (× several sizes)
each anchor → object? which class? how to shift it?

Before NMS



Sort by score; keep the best box; delete boxes overlapping it by IoU > 0.5; repeat.

After NMS (IoU threshold 0.5)



Left: three anchor shapes centred on one grid position. Right: non-maximum suppression (section 4.7) keeps the best box per object.

During training, each anchor gets a target by comparing it with the ground-truth boxes:

Model	Anchors per position	Positive if IoU ≥	Background if IoU <	In between
Faster R-CNN, stage 1 (RPN)	3 per FPN level	0.7	0.3	ignored
Faster R-CNN, stage 2	works on proposals	0.5	0.5	none
RetinaNet	9 (3 sizes × 3 shapes)	0.5	0.4	ignored
SSD300	4 or 6, by map	0.5	0.5	none

RetinaNet, SSD300 and Faster R-CNN's RPN also give each ground-truth box its single best-matching anchor even when the IoU is below the threshold, so no object is left without one.

4.5 Box regression: predict corrections, not coordinates

Instead of outputting pixel coordinates, the network predicts how to adjust the anchor. With anchor centre (x_a, y_a) , width w_a and height h_a , and object centre (x, y) with size (w, h) :

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \log \frac{w}{w_a}, \quad t_h = \log \frac{h}{h_a}$$

Dividing by the anchor size makes the targets similar for small and large objects. The logarithm makes "twice as wide" and "half as wide" equally large corrections. Faster R-CNN and SSD300 train these offsets with a **smooth L1** loss.

RetinaNet v2 uses a **GIoU** loss, which scores the overlap of the decoded box directly (GIoU = IoU minus a penalty for empty space in the smallest box enclosing both) and still gives a useful signal when the boxes don't overlap at all.

4.6 The imbalance problem, the thread through all three models

For an 800×1067 image, RetinaNet evaluates about **160,000 anchors** (five FPN levels with 9 anchors per position). A trash photo has a handful of objects, so well over 99.9% of anchors are background. If every anchor counted equally, the model could get a small loss by saying "background" everywhere and learning nothing about waste. Each model deals with this differently, and that difference is a big part of what you're comparing:

Model	Strategy	How
Faster R-CNN	Filter, then sample	The RPN keeps only about 2,000 candidate regions. Training then samples a fixed mix: 256 anchors per image at 50% positive for the RPN, and 512 regions at 25% positive for stage 2.
RetinaNet	Re-weight	Keeps every anchor but uses focal loss , which almost silences the easy background anchors (5.2).
SSD300	Mine the hard ones	Hard negative mining : of all background boxes, only the highest-loss ones count, 3 per positive.

4.7 Non-maximum suppression (NMS)

Neighbouring anchors all fire on the same object, so the raw output has many near-identical boxes. NMS removes them:

1. Drop boxes whose score is below a low threshold (0.01 in our setup).
2. For each class separately, sort the boxes by score.
3. Keep the top box. Delete every remaining box whose IoU with it is above 0.5.
4. Repeat with the next box still standing.

NMS runs **per class**, so an overlapping Can and Bottle don't suppress each other. There is a trade-off tied directly to the brief's "objects may overlap": if two bottles touch and their boxes overlap by more than the NMS threshold, one is deleted and becomes a missed detection. A lower threshold removes more duplicates but loses more real neighbours; a higher one does the opposite. We use 0.5 for all three models.

4.8 Feature Pyramid Network (FPN): small and large objects

Deep feature maps know *what* things are but are coarse. Shallow maps are sharp but understand little. An FPN adds a top-down path: it upsamples the deep, meaningful features and adds them (through 1×1 "lateral" convolutions) to the shallower, sharper maps. Every level of the pyramid ends up both sharp and meaningful. Objects are handled at

the level matching their size: small items on the high-resolution levels (strides 4–8), large items on the coarse ones.

SSD300 came before FPN. It predicts small objects from its highest-resolution map (38×38), but that map never receives the deep features from above. This, plus the 300×300 input, is why SSD300 is expected to struggle with cigarette butts and bottle caps.

4.9 The families of detectors

Family	Examples	Idea	Trade-off
Two-stage	Faster R-CNN , Mask R-CNN, Cascade R-CNN	Propose regions, then classify and refine each one	Accurate, slower
One-stage, anchor-based	RetinaNet , SSD (the YOLO family also sits here)	Classify and regress every anchor in one pass	Faster; must handle the imbalance
One-stage, anchor-free	FCOS, CenterNet	Predict from points or object centres, without anchors	Fewer settings to tune
Transformer	DETR, Deformable DETR	Predict a set of boxes directly; no NMS	Needs a lot of data and training time

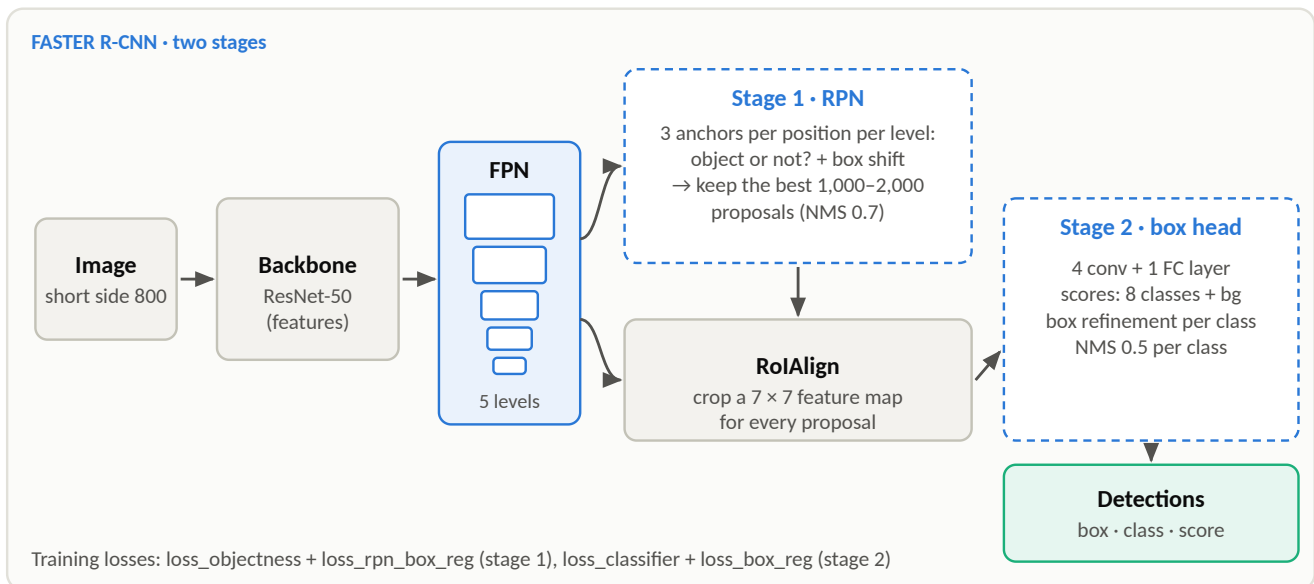
Be ready to explain

- Convert a box between VOC, COCO and YOLO formats (practise with figure 4.2)
- Compute an IoU by hand, and explain why small objects are harder to localise
- What an anchor is, and how anchors are labelled positive or negative
- Why detectors predict offsets (t_x, t_y, t_w, t_h) instead of coordinates
- The imbalance problem and each model's answer to it
- The NMS steps, and how the NMS threshold relates to overlapping objects
- What an FPN adds, and why SSD300 lacks it

5 The three models

Each section explains how the model works, what its training losses mean (these names appear in your training log), and its expected strengths and weaknesses. Each ends with a 30-second explanation you can give in the viva. All three are torchvision's COCO-pretrained versions.

5.1 Faster R-CNN (two-stage)



Faster R-CNN in torchvision's improved "v2" form. Stage 1 finds candidate regions; stage 2 classifies each one and refines its box.

1. **Backbone + FPN.** ResNet-50 and a feature pyramid produce five feature maps, from stride 4 (sharp) to stride 64 (coarse).
2. **Region Proposal Network (RPN).** At every position of every level there are 3 anchors: one size per level (32, 64, 128, 256 or 512 px) in three shapes (1:2, 1:1, 2:1). A small conv head predicts, for each anchor, "object or not" and 4 box offsets.
3. **Proposals.** The anchors are decoded into boxes, the best ones kept, and duplicates removed with NMS at IoU 0.7. About 2,000 proposals per image survive in training, 1,000 at test time.
4. **RoIAlign.** For every proposal, a fixed 7×7 patch of features is sampled from the matching FPN level. It uses bilinear sampling without rounding, which keeps boxes precise.
5. **Box head.** Four conv layers and one fully connected layer (the v2 design) turn each patch into 9 class scores (softmax) and 4 box offsets per class.
6. **Post-processing.** Per class: drop low scores, NMS at IoU 0.5, keep the top 100.

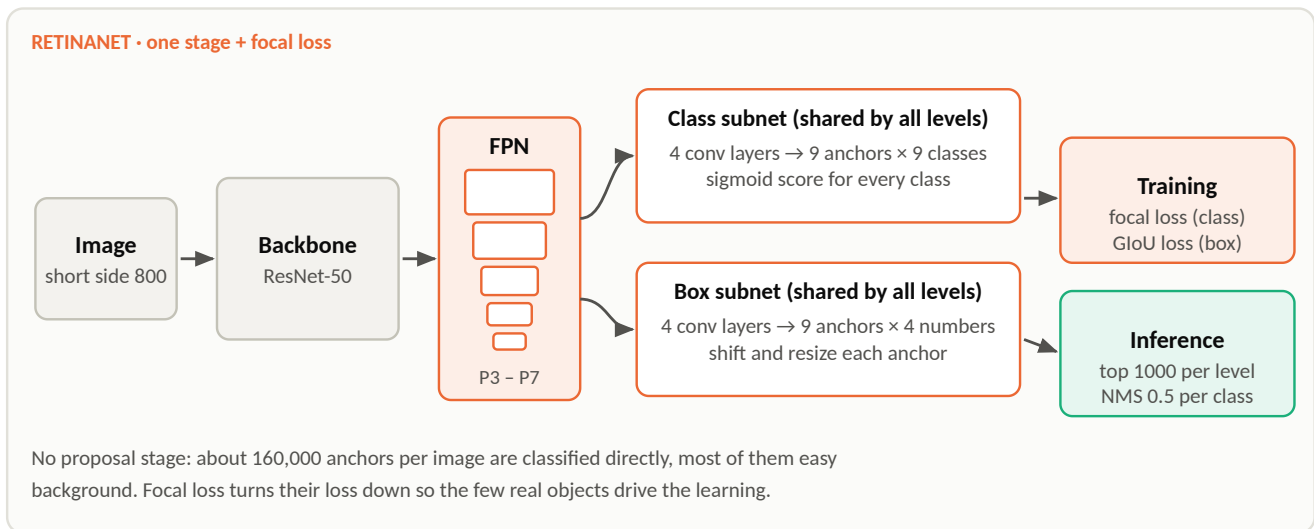
Loss in the log	What it measures
<code>loss_objectness</code>	RPN: is this anchor an object or background? (binary cross-entropy)
<code>loss_rpn_box_reg</code>	RPN: how far the anchor is from the object (smooth L1)
<code>loss_classifier</code>	Stage 2: which of the 9 classes (cross-entropy)
<code>loss_box_reg</code>	Stage 2: how far the proposal is from the object (smooth L1)

What v2 changes. A two-layer RPN head, conv + BatchNorm layers in the box head, and a longer training recipe with stronger augmentation (Li et al., 2021). It reaches 46.7 COCO mAP, against 37.0 for the original torchvision version.

Strengths: usually the most accurate; good on small objects (stride-4 level, 800-px input, and a second look at every region); robust with little data. **Weaknesses:** the slowest of the three, because up to 1,000 regions per image go through the box head; the largest model here.

In 30 seconds. "Faster R-CNN looks twice. First its region proposal network scores anchors all over the image and keeps about a thousand promising regions. Then, for each region, it crops features with RoIAlign and decides the class and the exact box. The second look makes it accurate, especially on small items, but slower."

5.2 RetinaNet (one-stage, focal loss)



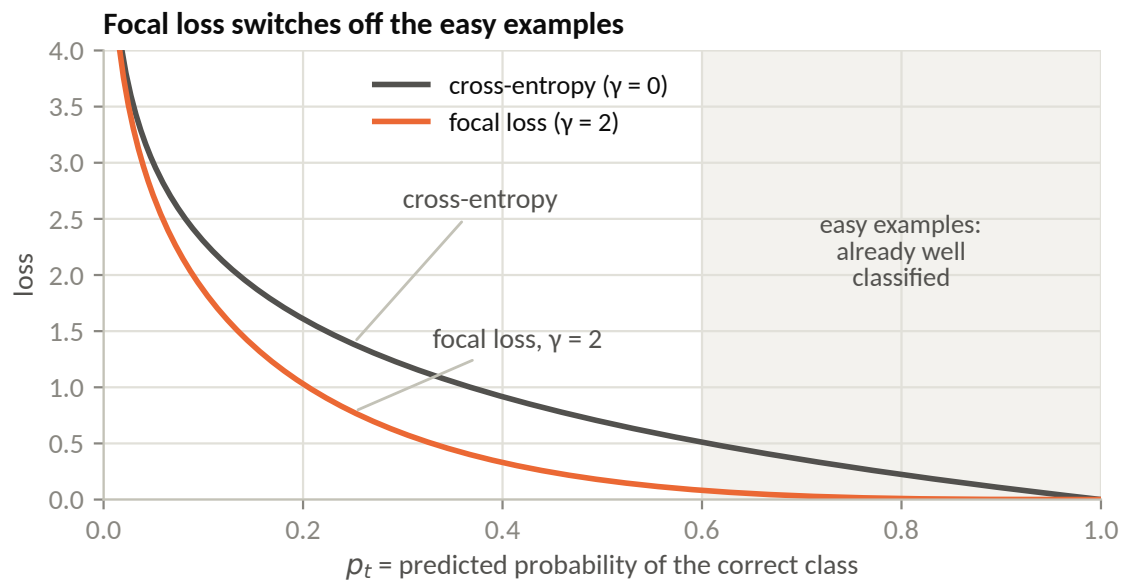
RetinaNet: one pass, two small subnets shared across all pyramid levels, and focal loss for the class scores.

1. **Backbone + FPN** with levels P3–P7 (strides 8 to 128).
2. **Anchors:** 9 per position (3 sizes × 3 shapes) on every level.
3. **Two subnets**, shared by all levels. The class subnet (4 conv layers) gives a separate sigmoid score for every class of every anchor. The box subnet (4 conv layers) gives 4 offsets per anchor.
4. **Inference:** on each level keep the 1,000 best anchors above the score threshold, decode them, and run NMS at 0.5 per class.

Focal loss

Let p_t be the probability the model gives to the *correct* answer. For a background anchor that is the probability of "not this class". Ordinary cross-entropy is $CE = -\log(p_t)$. Focal loss multiplies it by a factor that vanishes for easy examples:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t), \quad \gamma = 2, \quad \alpha = 0.25$$



Example (ignoring α)	p_t	Cross-entropy	$(1 - p_t)^2$	Focal loss	Loss reduced
Easy background anchor	0.99	0.0101	0.0001	0.000001	10,000 ×
Fairly easy	0.90	0.105	0.01	0.00105	100 ×
Hard, misclassified	0.10	2.303	0.81	1.865	1.2 ×

With 160,000 mostly easy anchors, the small cross-entropy losses of the easy ones still add up and drown out the few hard examples. Focal loss makes them negligible, so learning concentrates on real objects and confusing background. $\alpha = 0.25$ gives positives a fixed weight of 0.25 and negatives 0.75. With $\gamma = 2$ this balance works best in the RetinaNet paper.

Starting prior. The final class layer's bias starts at $b = -\log((1 - \pi)/\pi)$ with $\pi = 0.01$, so every anchor initially predicts about a 1% chance of an object. Starting at 50% would produce an enormous loss from 160,000 background anchors in the first iterations. Our code applies this initialisation when it replaces the last layer (chapter 10, Step 5).

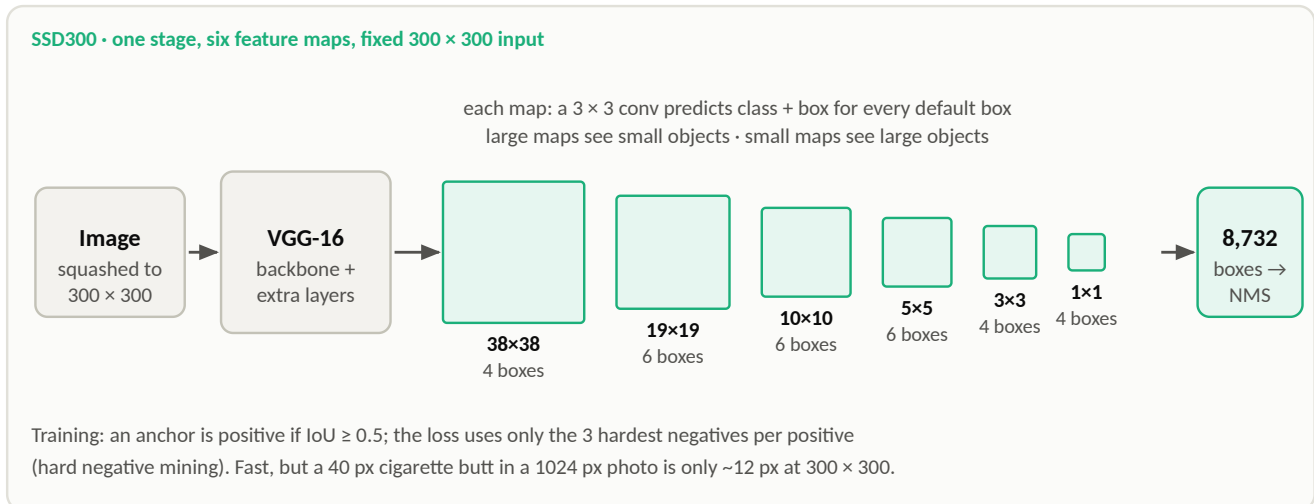
Loss in the log	What it measures
classification	Focal loss over all anchors, divided by the number of positive anchors
bbox_regression	GloU loss between decoded boxes and objects (positive anchors only)

What v2 changes. GroupNorm in the heads (stable with small batches), a GloU box loss and a longer training recipe. It reaches 41.5 COCO mAP, against 36.4 for v1.

Strengths: one-stage simplicity with accuracy close to two-stage models; copes well with lots of easy background such as streets and pavements. **Weaknesses:** evaluates many anchors, so it is slower than SSD300; its finest level is stride 8, one step coarser than Faster R-CNN's stride 4.

In 30 seconds. "RetinaNet is a one-stage detector. In one pass it scores and adjusts nine anchors at every position of five feature-map levels. Almost all anchors are easy background, so it uses focal loss, which multiplies each loss by one minus p_t , squared. Confident background contributes almost nothing, and the hard examples drive the learning."

5.3 SSD300 (one-stage, multi-scale, fast)



SSD300 predicts from six feature maps of decreasing size. Each map handles objects of one size range.

1. **Resize** the photo to exactly 300×300 . The aspect ratio is not preserved.
2. **VGG-16** plus extra layers that keep halving the size, giving six maps: 38×38 , 19×19 , 10×10 , 5×5 , 3×3 and 1×1 .
3. **Default boxes** (SSD's anchors): 4 or 6 per position, with sizes from 7% of the image on the largest map up to nearly the whole image on the smallest, and shapes 1:1, 2:1, 1:2 (plus 3:1, 1:3 on the middle maps). That is $38^2 \cdot 4 + 19^2 \cdot 6 + 10^2 \cdot 6 + 5^2 \cdot 6 + 3^2 \cdot 4 + 1^2 \cdot 4 = 8,732$ boxes.
4. **Predictors**: one 3×3 conv per map gives 9 class scores (softmax, including background) and 4 offsets for every default box.
5. **Training**: a default box is positive if $\text{IoU} \geq 0.5$. The loss is cross-entropy plus smooth L1, with **hard negative mining**: only the 3 highest-loss background boxes per positive count.

Loss in the log	What it measures
bbox_regression	Smooth L1 on offsets for positive default boxes
classification	Cross-entropy on positives plus the hardest negatives (3 : 1)

Strengths: by far the least computation (about 35 GFLOPs, against 152 and 280), a small checkpoint and simple inference. **Weaknesses:** small objects. A 40-px cigarette butt in a 1,024-px-wide photo is about 12 px at 300×300 , barely wider than one cell of the finest 38×38 map (about 8 px per cell). SSD300 also has no FPN, and it has the lowest COCO accuracy of the three (25.1).

In 30 seconds. "SSD shrinks the photo to 300 by 300 and, in one pass, predicts a class and a box correction for 8,732 default boxes spread over six feature maps. Large maps catch small objects and small maps catch large ones. It is fast and light, but at 300 by 300 a cigarette butt is only a few pixels wide, so small items are its weak spot."

5.4 Side by side

	Faster R-CNN v2	RetinaNet v2	SSD300
Family	Two-stage	One-stage	One-stage
Backbone	ResNet-50 + FPN (5 levels)	ResNet-50 + FPN (P3–P7)	VGG-16 + extra layers
Input size	Short side 800 px	Short side 800 px	Fixed 300×300

	Faster R-CNN v2	RetinaNet v2	SSD300
Anchors	3 per position per level (RPN)	9 per position per level	4–6 per position; 8,732 total
Handles background by	Proposals + sampling	Focal loss	Hard negative mining 3:1
Class scores	Softmax over 9	Sigmoid per class	Softmax over 9
Box loss	Smooth L1	GIoU	Smooth L1
COCO box mAP*	46.7	41.5	25.1
Parameters, COCO → ours	43.7 M → 43.3 M	38.2 M → 36.5 M	35.6 M → 24.7 M
Compute per image*	280 GFLOPs	152 GFLOPs	35 GFLOPs
Our checkpoint size	≈ 174 MB	≈ 146 MB	≈ 99 MB
Expected strength	Accuracy, small objects	Balance of speed and accuracy	Speed, size
Expected weakness	Slowest	Many anchors; no stride-4 level	Small objects

* Figures published by torchvision for the COCO-trained weights. Parameter counts drop with our 9-class head because the class layers shrink. SSD300 shrinks most because its class predictors are large convolutions whose size grows with the number of classes. Your measured speed goes in the report (chapter 8).

Be ready to explain

- The 30-second explanation of each model, without notes
- What the RPN outputs, and what RoIAlign is for
- Focal loss: formula, γ and α , and the worked numbers in the table
- Why RetinaNet's final bias starts at a 1% prior
- Where SSD300's 8,732 boxes come from, and why it struggles with small items
- Softmax (Faster R-CNN, SSD) vs per-class sigmoid (RetinaNet)
- Which model you'd expect to win on accuracy and which on speed, and why

6 Data: formats, cleaning and augmentation

Bad data can't be rescued by a good model. This chapter covers what's in the download, what we use and why, how the labels are cleaned and checked, how the split is made, and how augmentation works for detection.

6.1 What's in the download, and what we use

The Kaggle dataset has 163 labelled photos of street-collected trash in 8 classes, plus derived copies. According to the dataset page it contains the originals, the same 163 resized to 1024×1024 , 4,923 augmented images (made with the Albumentations library), a ready-made 70 / 20 / 10 train / validation / test split, and labels in Pascal VOC, COCO and YOLO formats. The licence is Apache 2.0.

Part of the download	Use it?	Why
Original photos + VOC labels	Yes	The real data. We make our own clean copy and our own split.
Resized 1024×1024 copies	No	Forcing non-square photos into a square distorts shapes. Our clean copy resizes but keeps the aspect ratio.
4,923 augmented images	No	Transformed copies of the same 163 photos. We can't tell which original each came from, so they risk leakage. On-the-fly augmentation replaces them.
Ready-made split	No	We can't verify it is leak-free, and its test set would be only about 16 photos.
COCO and YOLO labels	No	Same boxes, different format. VOC already matches torchvision's box layout.

If the examiner asks "why didn't you use the augmented data?", that table is your answer. It shows you thought about leakage, which is a mark of maturity.

6.2 The three label formats, in the files

The same (illustrative) Bottle Cap in a 1600×1200 photo, as it appears in each format:

Pascal VOC · `img_012.xml`

```
<annotation>
  <filename>img_012.jpg</filename>
  <size>
    <width>1600</width>
    <height>1200</height>
  </size>
  <object>
    <name>Bottle Cap</name>
    <bndbox>
      <xmin>812</xmin>
      <ymin>455</ymin>
      <xmax>871</xmax>
      <ymax>509</ymax>
    </bndbox>
  </object>
</annotation>
```

COCO · `one .json` for all images

```
{
  "images": [
    {
      "id": 12,
      "file_name": "img_012.jpg",
      "width": 1600,
      "height": 1200
    }
  ],
  "annotations": [
    {
      "id": 1,
      "image_id": 12,
      "category_id": 2,
      "bbox": [812, 455, 59, 54],
      "area": 3186,
      "iscrowd": 0
    }
  ],
  "categories": [
    {
      "id": 2,
      "name": "Bottle Cap"
    }
  ]
}
```

YOLO · `img_012.txt`

```
1 0.5259 0.4017 0.0369 0.0450
```

One line per object: class index (counting from 0), then centre x, centre y, width and height as fractions of the image size:

$cx = (812 + 871) / 2 / 1600 = 0.5259$

$cy = (455 + 509) / 2 / 1200 = 0.4017$

$w = 59 / 1600 = 0.0369$

$h = 54 / 1200 = 0.0450$

COCO's `bbox` is x, y, width, height, so $871 - 812 = 59$ and $509 - 455 = 54$.

6.3 Cleaning the labels (notebook Step 2)

Phone rotation (EXIF). Phones often save the pixels in the sensor's orientation and add a note: "rotate 90° when you display this". Photo viewers obey the note; some labelling tools and libraries don't. If the boxes were drawn on the rotated view but your code reads the raw pixels (or the reverse), every box is in the wrong place, and the model learns nonsense without any error message. The notebook compares the width $\times$ height written in each XML with both versions of the image and uses the one that matches.

Labels drawn on a resized copy. If the XML size matches neither version, the boxes are rescaled by actual size ÷ recorded size.

Resize. The clean copy has its long side at most 1,024 px, with the aspect ratio kept and the boxes scaled by the same factor. Loading is faster, and Faster R-CNN and RetinaNet resize to an 800-px short side internally anyway.

Invalid boxes. Boxes are clipped to the image, and any box under 2 px wide or tall is dropped. A zero-size box crashes torchvision training with "All bounding boxes should have positive height and width".

Class names. Spelling variants such as `bottle_cap`, `Bottle-Cap` and `bottle cap` are normalised to `Bottle Cap`, so a typo can't create a ninth class.

6.4 Exploring the data (Step 3)

Look at	Because	Write in the report
Boxes per class	Rare classes get fewer lessons and unstable test scores	The class table; name the rarest classes
Object size, $\sqrt{(\text{box area} \div \text{image area})}$	Small objects (under 0.1) are the hard cases, especially for SSD300	Share of small objects and which classes they are
Boxes per image	Crowded photos mean overlaps and NMS risk	Median and maximum
Boxes drawn on photos	Catches rotated, shifted or swapped labels before you waste a training run	One sentence: "labels verified visually"

6.5 Splitting (Step 4)

The split is **by photo**: all boxes of one photo go to the same set. 70 / 15 / 15 of 163 gives 114 / 24 / 25 photos. A purely random split could leave a rare class with no test examples, which would make its AP meaningless. So the notebook tries seeds 0, 1, 2, ... and keeps the first split where **every class has at least 2 boxes in both validation and test** (falling back to 1 if that's impossible). The split and its seed are saved in `split.json`. Every later run and every teammate uses the same file, so your numbers are comparable.

6.6 Augmentation for detection (Step 5)

In image classification you can transform a picture however you like. In detection, **the boxes must be transformed with the pixels**: flip the image and every xmin, xmax must be mirrored too. torchvision's `transforms.v2` does this automatically when the boxes are wrapped as `BoundingBoxes`.

Transform	How often	What it simulates
<code>RandomPhotometricDistort</code>	50%	Different light, camera and colour: brightness, contrast, saturation, hue
<code>RandomZoomOut</code> (up to 1.5×)	30%	Objects further away: the photo shrinks inside a larger canvas, so objects get smaller
<code>RandomIoUCrop</code>	30%	Objects closer up: a crop that keeps most of each object
<code>RandomHorizontalFlip</code>	50%	Mirror image; trash has no left or right
<code>SanitizeBoundingBoxes</code>	always	Removes boxes a crop has cut away (and their labels)
<code>ToDtype(float32, scale=True)</code>	always	Pixel values 0–255 → 0–1, which the models expect

What we avoid. Large rotations: an axis-aligned box around a rotated object must grow, so the labels become loose. Vertical flips too, unless your photos are taken straight down; look at the data and decide. Validation and test images are **never** augmented.

6.7 Class imbalance

If one class has five times more boxes than another, the model sees it five times more often. We don't correct for this. Instead we **report per-class results** so the effect is visible, which is the honest approach for a comparison study. If a class turns out to be very weak, possible remedies for the future-work section are: oversample photos that contain it, photograph and label more examples, or use a class-weighted loss.

Be ready to explain

- Which parts of the download you used and why you skipped the augmented set
- What EXIF orientation is and how it can silently break a detector
- Why the split is made by photo, and how you made sure every class appears in validation and test
- Why boxes must be transformed together with the image, with an example (horizontal flip)
- What `SanitizeBoundingBoxes` does after a crop

7 Training

One training loop serves all three models. Only the model-building function changes. This chapter gives the full recipe, what happens in a single iteration, and how to read the training curves.

7.1 The recipe

Setting	Value	Reason
Starting weights	COCO (torchvision DEFAULT)	Transfer learning (D1)
New layer	9 outputs (8 + background)	Our classes
Optimizer	AdamW, lr 1e-4, wd 1e-4	Robust with little tuning, so one setting fits all three
Schedule	1-epoch warm-up, then cosine to 0	Gentle start for a random new layer; fine steps at the end
Epochs, batch	30 epochs, batch 4	About 870 updates; fits T4 memory with mixed precision
Stability	Gradient clipping at norm 10; loss checked for <code>nan</code>	One bad batch can't wreck the run
Speed	Mixed precision on GPU	About 2× faster on a T4
Input size	800 px short side / 300 × 300	Each model's native design
Checkpoint kept	Epoch with the best validation mAP@0.5:0.95	Early stopping without guessing the epoch count
Seed	42	Repeatable runs

Why the same learning rate for all three? AdamW adapts the step size for every weight, so it is far less sensitive to the exact learning rate than plain SGD, and 1e-4 is a standard fine-tuning value. If you have spare time, a fairer-still extension is to try 5e-5, 1e-4 and 2e-4 for *each* model and keep the best by validation mAP. Every model gets the same tuning budget.

7.2 One iteration, in code order

1. The DataLoader returns a *list* of 4 images (sizes may differ) and a list of 4 target dicts (`boxes` , `labels`).
2. Move them to the GPU.
3. Forward pass in **training mode**. torchvision detectors then return a *dict of losses* instead of boxes.
4. Add the losses into one number, and stop if it isn't finite.
5. Zero the old gradients; backpropagate (scaled, for mixed precision).
6. Unscale the gradients, clip them to norm 10, and take the optimizer step.
7. Advance the learning-rate schedule by one step.

After each epoch the model switches to **evaluation mode** (which returns boxes) and predicts on the validation set. Then it computes COCO mAP, saves the checkpoint if this is the best epoch so far, and writes the history to `results/history_<model>.json` .

7.3 Reading the training curves

Step 6 plots two panels for all models: training loss divided by its epoch-1 value, and validation mAP. The loss is shown **relative to epoch 1** because each model defines its loss differently. A RetinaNet loss of 0.4 and an SSD loss of 3.0 can't be compared directly; their trends can.

Pattern	What you see	Meaning	Action
Healthy	Loss falls fast, then slowly; val mAP rises, then flattens	Learning well	None
Overfitting	Loss keeps falling; val mAP peaks, then drops	Memorising the training photos	The best-epoch checkpoint already protects you; say so in the report
Under-trained	Val mAP still rising at epoch 30	More epochs would help	Raise <code>EPOCHS</code> to 40–50 <i>for all three</i> and retrain
Unstable	Loss jumps around or becomes <code>nan</code>	Step too big, fp16 overflow, or bad boxes	<code>USE_AMP = False</code> , <code>LR = 5e-5</code> , check the labels
Stuck	Val mAP ≈ 0 after 5+ epochs	A pipeline bug, almost always the labels	Chapter 13

7.4 Time and memory on a Colab T4

Model	Approx. time for 30 epochs	Batch 4 memory
Faster R-CNN v2	15–25 min	fits with mixed precision
RetinaNet v2	15–20 min	fits with mixed precision
SSD300	about 5 min	small

These are estimates; the free T4 varies. If you see "CUDA out of memory", set `BATCH_SIZE = 2` and restart the runtime. Keep the Colab tab open while training. If the session drops, re-run the notebook: finished models are skipped, and an interrupted model retrains from its first epoch.

Be ready to explain

- Every row of the recipe, and why it's the same for all models
- What a torchvision detector returns in training mode vs evaluation mode
- Why the loss chart is relative to epoch 1
- How you picked the epoch to keep, and why on validation
- What overfitting would look like in your curves, and whether you saw it

8 Evaluation metrics

The brief asks for precision, recall and mAP. You need to be able to compute each one by hand from a small example. This chapter builds them up from one rule: how a prediction is matched to an object.

8.1 Matching predictions to objects

A prediction is a **true positive (TP)** when all three hold:

1. its class equals the class of a ground-truth box,
2. its IoU with that box is at least 0.5, and
3. that box hasn't already been claimed by a higher-scoring prediction.

Predictions are matched in order of confidence, highest first. This is the PASCAL VOC and COCO rule, and it is exactly what `match_image` in the notebook does. Every other prediction is a **false positive (FP)**. Every ground-truth box nobody claimed is a **false negative (FN)**, a missed object.

Why there are no true negatives. A true negative would be "a background region correctly left without a box". An image contains countless possible regions, so they can't be counted. That's why detection doesn't use accuracy and relies on precision and recall instead.

8.2 Precision, recall and F1

$$\text{Precision} = \frac{TP}{TP+FP} \quad \text{Recall} = \frac{TP}{TP+FN} \quad F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

Precision: of the boxes we drew, how many were right? **Recall:** of the objects in the photos, how many did we find? **F1** is their harmonic mean. It is only high when both are high.

On a sorting line, low recall means recyclables slip past unsorted. Low precision means the robot grabs things that aren't there, or puts them in the wrong bin. Which matters more depends on the plant, and saying so in your conclusion shows judgement.

8.3 The confidence threshold

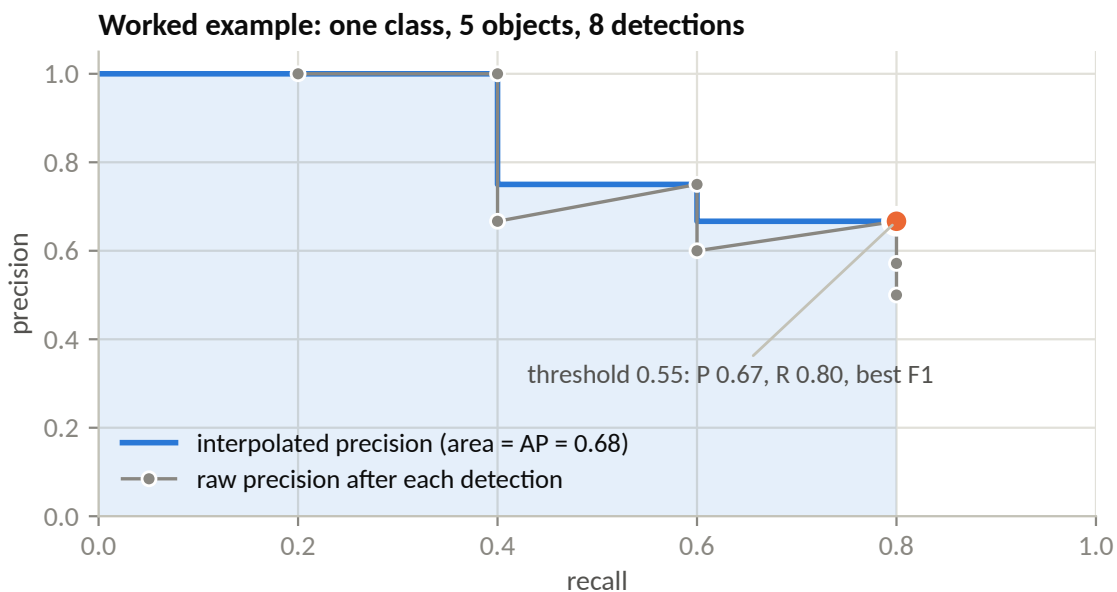
Every box has a score. Raise the threshold and you keep fewer, surer boxes: precision rises, recall falls. In the worked example below, a threshold of 0.85 gives $P = 1.00$ and $R = 0.40$. At 0.55 it gives $P = 0.67$ and $R = 0.80$ ($F1 = 0.73$), and at 0.2 it gives $P = 0.50$ with R still 0.80.

Scores from different models aren't on the same scale; focal-loss models, for example, tend to give lower scores. One fixed threshold for all three would therefore favour some models. So the notebook picks each model's threshold where **F1 is highest on the validation set** (trying 0.05 to 0.95), then reports precision, recall and F1 on the test set at that threshold. **Always state the threshold next to P and R.**

8.4 Precision–recall curve and Average Precision (AP)

Instead of one threshold, sweep all of them. Sort the detections by score and, after each one, compute precision and recall so far. Worked example: one class, 5 objects in the images, 8 detections.

Rank	Score	Match	Cum. TP	Cum. FP	Precision	Recall
1	0.95	TP	1	0	1.00	0.20
2	0.91	TP	2	0	1.00	0.40
3	0.84	FP	2	1	0.67	0.40
4	0.80	TP	3	1	0.75	0.60
5	0.62	FP	3	2	0.60	0.60
6	0.55	TP	4	2	0.67	0.80
7	0.40	FP	4	3	0.57	0.80
8	0.21	FP	4	4	0.50	0.80



The raw precision (grey) zig-zags. AP uses the **interpolated** precision (blue): at each recall level, the best precision reached at that recall or higher. The shaded area is the AP.

AP is the area under the interpolated curve, summed over the points where recall increases:

$$AP = \sum_k (r_k - r_{k-1}) \cdot p_{\text{interp}}(r_k) = 0.2 \cdot 1.00 + 0.2 \cdot 1.00 + 0.2 \cdot 0.75 + 0.2 \cdot 0.67 = 0.68$$

One object was never found, so recall stops at 0.8 and AP can't exceed 0.8. That's how misses lower AP. COCO samples the curve at 101 fixed recall points (0, 0.01, ..., 1) instead of at every step. The numbers differ only slightly: when the notebook was tested, its own AP@0.5 and the official COCO-style value agreed to within 0.001.

8.5 The mAP family

AP is computed per class. **mAP** is the mean over the classes.

Metric	Definition	Tells you
mAP@0.5	mAP with IoU ≥ 0.5 counting as a match (the VOC metric)	Does it find the objects?
mAP@0.75	mAP with the stricter IoU ≥ 0.75	Are the boxes tight?
mAP@0.5:0.95	Average of mAP at IoU 0.50, 0.55, ..., 0.95 (10 thresholds). The main COCO metric.	Detection <i>and</i> box quality, in one number

Metric	Definition	Tells you
AR@100	Recall averaged over IoU 0.5–0.95, allowing up to 100 boxes per image	How many objects can be found at all
mAP small / medium / large	COCO sizes: area < 32 ² px, 32 ² –96 ² , > 96 ² (in our ≤1024-px images). –1 means no objects of that size.	Where the misses are

What to headline: mAP@0.5:0.95, because the brief asks about the "quality of bounding-box predictions", together with mAP@0.5, plus precision, recall and F1 at the chosen threshold. mAP@0.5:0.95 is always lower than mAP@0.5 because it also demands tight boxes. Be ready to explain that gap.

8.6 The confusion matrix for detection

A classification confusion matrix only has classes. A detection matrix adds **background** as row and column 0. Each prediction above the threshold is paired with the object it overlaps most (IoU ≥ 0.5), whatever its class.

- **Diagonal:** right class, right place.
- **Off-diagonal in a class row:** right place, wrong class. This is where you see look-alike classes.
- **Background column** (predicted = background): objects that no box covered, i.e. misses.
- **Background row** (true = background): boxes on nothing, i.e. false alarms.

In the notebook's chart the numbers are counts and the shading shows each row's share, so rare classes stay readable next to the large background row.

8.7 Speed and size

Latency is the median time for one image (batch 1) on the T4 with mixed precision, after 3 warm-up runs. It includes the model's own resizing, the network and NMS, but not reading the JPEG from disk. **FPS** = 1000 ÷ latency in ms. GPU work is asynchronous, so the timer calls `torch.cuda.synchronize()` before and after each run; otherwise you'd only time the launch. **Parameters** is the number of weights, and **size** is the checkpoint file in MB (32-bit floats). A laptop CPU will be many times slower. Mention it if you demo live.

8.8 How much to trust the numbers

25 test photos is a small sample. One object can move a class's AP by 10 points or more. Write "on our 25-image test set". Base your conclusions on clear gaps and on patterns that repeat across metrics, classes and object sizes. A good future-work item: a bootstrap confidence interval (re-sample the test photos 1,000 times and report the spread of mAP), or k-fold cross-validation.

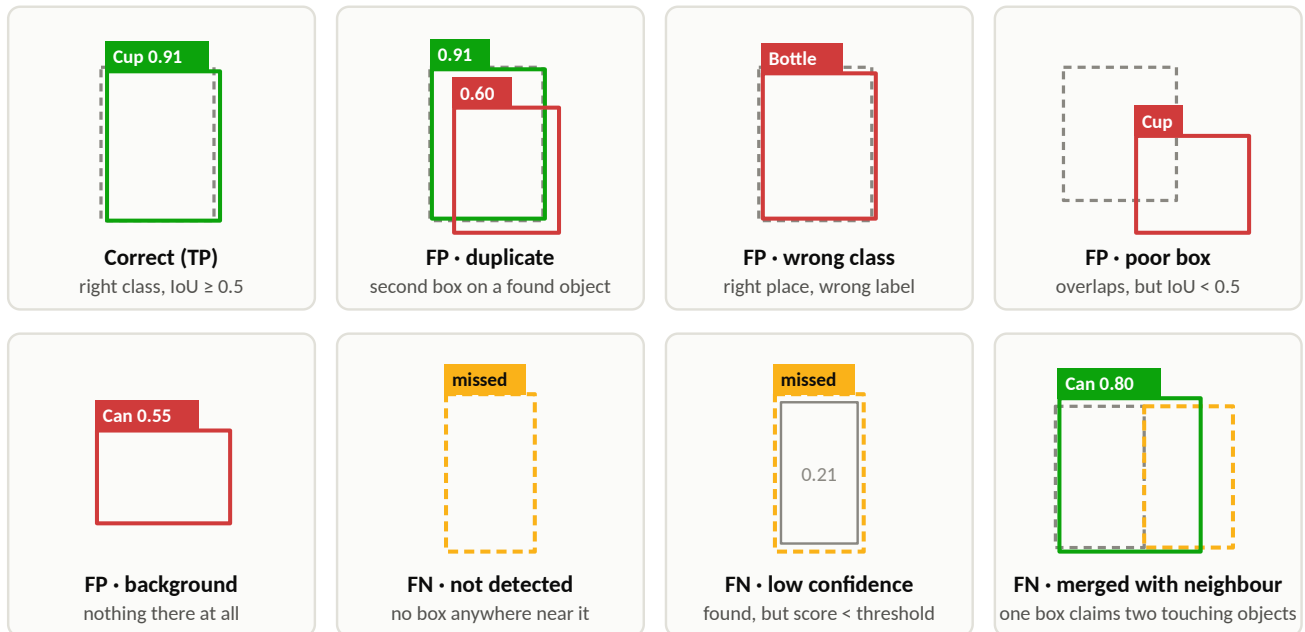
Be ready to explain

- The three matching conditions, and why detection has no true negatives
- Compute precision, recall and F1 from TP / FP / FN counts
- How the threshold trades precision against recall, and how you chose yours
- Rebuild the worked AP example from the table
- mAP@0.5 vs mAP@0.5:0.95, and why the second is lower
- How to read each part of the detection confusion matrix
- Why the timing code calls `torch.cuda.synchronize()`

9 Error analysis

Metrics say *how much* each model gets wrong. Error analysis says *what kind* of mistakes and *why*, and that is what the brief means by "analysis of correct detections, missed detections and false detections". It is also where you can show real understanding.

9.1 The error types



Grey dashed = the true object. Green = correct box. Red = false detection. Yellow dashed = missed object.

The notebook gives every false detection and every missed object exactly one reason. The checks run top to bottom, and the first rule that fits wins. "Near" means $\text{IoU} \geq 0.1$; a match means $\text{IoU} \geq 0.5$.

False detection (FP)	Rule	Missed object (FN)	Rule
duplicate	Matches a same-class object ($\text{IoU} \geq 0.5$) that a better box already claimed	merged with neighbour	A kept same-class box matches it, but was claimed by a touching object
wrong class	$\text{IoU} \geq 0.5$ with an object of another class	low confidence	A same-class box matches it, but scored below the threshold
poor box	Near an object ($0.1 \leq \text{IoU} < 0.5$)	wrong class	A kept box of another class covers it ($\text{IoU} \geq 0.5$)
background	$\text{IoU} < 0.1$ with every object	poor box	The best kept box is only near it ($\text{IoU} 0.1\text{--}0.5$)
		not detected	Nothing near it at all

One mistake can appear on both sides. A Cup labelled as Bottle is a false Bottle (FP: wrong class) *and* a missed Cup (FN: wrong class). That's how standard counting works, so mention it when you present the table. The FP categories follow the TIDE toolbox (Bolya et al., 2020), in simplified form.

9.2 From error type to cause to remedy

Error	Likely cause in this dataset	Remedy (future work)
FP · duplicate	Two boxes on one long or crumpled object both survive NMS	Tune the NMS threshold; more training data
FP / FN · wrong class	Look-alike classes: Cup vs Bottle vs Plastic; Cigarette Box vs Cardboard	More examples of the confused pairs; clearer class definitions
FP / FN · poor box	Small, partly hidden or crushed objects; a few pixels of error matter (figure 4.3)	Higher input resolution; tighter labels
FP · background	Leaves, stones, shadows, paper scraps that look like waste	Add photos of empty ground; more varied backgrounds
FN · merged	Touching objects of the same class; NMS deletes the second box	Higher NMS threshold or Soft-NMS; relates to "objects may overlap"
FN · low confidence	The model saw it but wasn't sure	Lower threshold (costs precision); more data for that class
FN · not detected	Tiny, hidden or unusual objects; rare classes	Higher resolution; tiling large photos; more data

9.3 Recall by object size

The notebook splits test objects by $\sqrt{(\text{box area} \div \text{image area})}$: **small** under 0.1, **medium** 0.1–0.3, **large** above 0.3. It then reports recall in each group. Reading it:

- If **all three** models miss small objects, it's a data problem: too few small examples, or too little resolution in the photos themselves.
- If **only SSD300** drops sharply on small objects, it's the architecture: a 300×300 input and no FPN. That tests hypothesis H2 directly.

9.4 Choosing example images

The notebook saves side-by-side panels (one column per model) for the three hardest test photos and the easiest one. Green boxes are correct, red are false and yellow dashed are missed. Use two hard photos, one easy photo and one of your own unseen photos in the report, each with a one-sentence caption naming the error types you see.

9.5 Writing an observation

Every observation should run **evidence → cause → implication**. A template, not a result:

"On the 25 test photos, SSD300 found __ of __ small objects (recall __) against __ for Faster R-CNN. Most of its misses were not detected on Cigarette Butt and Bottle Cap. This fits its 300×300 input, where a 40-px butt shrinks to about 12 px, and its lack of an FPN. For a sorting line that handles small litter, this makes SSD300 unsuitable despite being __x faster."

Fill in the blanks from your own tables. If a result contradicts a hypothesis from section 2.5, say so plainly and suggest why. That's a strength in a report, not a weakness.

Be ready to explain

- The error types and their IoU rules
- Why one mistake can count as both an FP and an FN
- Your most common error type for each model, and your explanation for it
- What recall by object size shows about SSD300
- How "merged with neighbour" connects to the brief's "objects may overlap"

10 Build guide, step by step

This chapter follows the notebook `trash_detection_colab.ipynb` cell by cell. The code shown is copied directly from the tested notebook. For each step you get the goal, the key code, what the important lines do, and what to check before moving on.

Getting started. Upload the notebook to Google Drive and open it with Google Colab. Choose **Runtime → Change runtime type → T4 GPU**. Run the cells from the top. You need to edit exactly one line (`ORIG_DIR` , in Step 2). The only other changes you might make are `TRAIN_THESE` in Step 6, if you split training across teammates, and `BATCH_SIZE` if the GPU runs out of memory.

Step 0 — Settings, Drive and folders

5 minutes · everyone

All training settings sit in one cell at the top, identical for all three models. The next cells install the COCO metric tools, mount Google Drive and create the project folders. `MyDrive/trash_detection/` keeps everything permanent: the zip, `split.json`, checkpoints and results. `/content/work` is Colab's fast local disk, rebuilt every session.

```
# ---- Settings (identical for all three models) ----
EPOCHS = 30                # passes over the training images
BATCH_SIZE = 4             # images per step; use 2 if you get "CUDA out of memory"
LR = 1e-4                  # AdamW learning rate
WEIGHT_DECAY = 1e-4
WARMUP_EPOCHS = 1         # learning rate ramps up during the first epoch
MIN_SIZE, MAX_SIZE = 800, 1333 # Faster R-CNN / RetinaNet input (SSD300 is always 300 x 300)
SEED = 42
PRETRAINED = True         # start from COCO-trained weights (transfer learning)
```

Check: the output says `device: cuda` and names a Tesla T4. If it says `cpu`, the GPU runtime isn't on.

Step 1 — Download the dataset

5 minutes · data lead (everyone once)

```
ZIP_PATH = PROJECT_DIR / "trash_dataset.zip"
DATA_URL = ("https://www.kaggle.com/api/v1/datasets/download/"
            "pandapowr/labelled-trash-dataset-pascal-voc-coco-and-yolo")
RAW_DIR = WORK_DIR / "raw"

if not ZIP_PATH.exists():
    subprocess.run(["curl", "-L", "--fail", "-o", str(ZIP_PATH), DATA_URL], check=True)
if not zipfile.is_zipfile(ZIP_PATH):
    ZIP_PATH.unlink()
    raise RuntimeError("The download is not a zip file. Use the browser download described above.")
if not RAW_DIR.exists():
    with zipfile.ZipFile(ZIP_PATH) as z:
        z.extractall(RAW_DIR)
print("Unzipped to", RAW_DIR)
```

- Kaggle's direct download link normally works for public datasets without signing in, which respects the "do not sign in" rule. The zip is saved in Drive once, so later sessions skip the download.
- `zipfile.is_zipfile` catches the case where Kaggle returns an error page instead of the data. In that case, download the zip in your browser, upload it to `MyDrive/trash_detection/`, rename it `trash_dataset.zip` and re-run the cell.
- The next cell, `show_tree`, prints every folder with a count of its file types. Use it to find the folder with the original data.

Step 2 — Read the labels and make a clean copy

10 minutes · data lead

Set `ORIG_DIR` to the folder that holds the **163 original images and their 163 .xml files** (images and XMLs may sit in sibling sub-folders). Then the code reads every VOC file and pairs it with its image:

```
def read_voc(xml_path):
    """Read one Pascal VOC file: the image size, and a class name + [xmin, ymin, xmax, ymax]
    for every object."""
    root = ET.parse(xml_path).getroot()
    size = root.find("size")
    wh = (0, 0)
    if size is not None:
        wh = (int(float(size.findtext("width", "0"))), int(float(size.findtext("height", "0"))))
    names, boxes = [], []
    for obj in root.iter("object"):
        bb = obj.find("bndbox")
        names.append(clean_name(obj.findtext("name")))
        boxes.append([float(bb.findtext(k)) for k in ("xmin", "ymin", "xmax", "ymax")])
    return root.findtext("filename", ""), wh, names, boxes
```

```
def load_records(folder):
    """Pair every .xml with its image (same file name, any sub-folder)."""
    folder = Path(folder)
    images = {p.stem: p for p in folder.rglob("*") if p.suffix.lower() in IMG_EXTS}
    records, missing = [], []
    for xml_path in sorted(folder.rglob("*.xml")):
        fname, xml_wh, names, boxes = read_voc(xml_path)
        img = images.get(xml_path.stem) or images.get(Path(fname).stem)
        if img is None:
            missing.append(xml_path.name)
            continue
        records.append({"id": xml_path.stem, "src": str(img), "xml_wh": xml_wh,
                        "names": names, "boxes": boxes})
    return records, missing
```

Then each photo is made upright, resized to at most 1,024 px, and its boxes are adjusted to match (section 6.3 explains why):

```
def make_clean_copy(rec):
    with Image.open(rec["src"]) as im:
        raw = im.convert("RGB")
        upright = ImageOps.exif_transpose(im).convert("RGB")
    xw, xh = rec["xml_wh"]
    img, note = upright, "ok"
    if (xw, xh) == raw.size and raw.size != upright.size:
        img, note = raw, "labels match the raw (unrotated) pixels"
    W, H = img.size
    boxes = np.array(rec["boxes"], dtype=np.float32).reshape(-1, 4)
    if xw > 0 and xh > 0 and (xw, xh) != (W, H):
        # labels were drawn on a resized copy
        boxes *= np.array([W / xw, H / xh, W / xw, H / xh], dtype=np.float32)
        note = "boxes rescaled"
    scale = min(1.0, MAX_SIDE / max(W, H))
    if scale < 1.0:
        img = img.resize((round(W * scale), round(H * scale)), Image.BILINEAR)
        boxes *= scale
    W, H = img.size
    boxes[:, [0, 2]] = boxes[:, [0, 2]].clip(0, W)
    boxes[:, [1, 3]] = boxes[:, [1, 3]].clip(0, H)
    # drop empty or 1-pixel boxes
    keep = ((boxes[:, 2] - boxes[:, 0]) >= 2) & ((boxes[:, 3] - boxes[:, 1]) >= 2)
    out = CLEAN_DIR / f"{rec['id']}.jpg"
    img.save(out, quality=95)
    return {"id": rec["id"], "image": str(out), "size": (W, H),
            "names": [n for n, k in zip(rec["names"], keep) if k],
            "boxes": boxes[keep].round(1).tolist(), "note": note, "dropped": int((~keep).sum())}
```

- `ImageOps.exif_transpose` applies the phone's rotation flag. The code uses the raw pixels only if the XML's recorded size matches them and not the rotated version.

- If the XML size matches neither version, the labels were drawn on a resized copy, so the boxes are multiplied by the size ratio.
- `scale` shrinks large photos, and the boxes are multiplied by the same factor, so they still fit.
- `keep` drops boxes under 2 px after clipping. The printed notes tell you how many photos needed each fix.

Check: about 163 labelled images; "label files without an image" is 0 or close to it. If you get 4,000+ images, you picked the augmented folder.

Step 3 — Explore and verify

15 minutes · data lead, results shared with everyone

```
CLASSES = sorted({n for d in data for n in d["names"]})
print(len(CLASSES), "classes:", CLASSES) # expect 8; if you see 9, fix the spelling in clean_name
CLASS_TO_ID = {c: i + 1 for i, c in enumerate(CLASSES)} # 0 is reserved for "background"
NUM_CLASSES = len(CLASSES) + 1 # 8 waste classes + background = 9
```

- Class ids start at 1 because torchvision reserves 0 for background. That's why `NUM_CLASSES` is 9.
- The cell prints boxes and images per class and the boxes per image, and saves `eda.png` (class counts and object sizes) and `label_check.png` (8 photos with their boxes drawn).

Check: exactly 8 classes are printed. If there are 9, a spelling variant slipped through; extend `clean_name`. **Open `label_check.png`**: every box must sit tightly on its object. If boxes are rotated or shifted, stop and see chapter 13.

Step 4 — Split 70 / 15 / 15

2 minutes · data lead; share `split.json` with the team

```
def make_split(data, fracs=(0.70, 0.15, 0.15), min_boxes=2, tries=2000):
    """Random split by image. Tries seeds 0, 1, 2, ... until every class has at least
    min_boxes boxes in val AND in test."""
    n = len(data)
    n_tr, n_va = round(fracs[0] * n), round(fracs[1] * n)
    ids = sorted(d["id"] for d in data)
    names = {d["id"]: d["names"] for d in data}
    for need in (min_boxes, 1):
        for seed in range(tries):
            order = ids[:]
            random.Random(seed).shuffle(order)
            parts = {"train": order[:n_tr], "val": order[n_tr:n_tr + n_va],
                    "test": order[n_tr + n_va:]}
            ok = all(sum(names[i].count(c) for i in parts[s]) >= need
                    for s in parts for c in CLASSES)
            if ok:
                return {"seed": seed, "min_boxes": need, **parts}
    raise RuntimeError("No seed puts every class in all three splits")
```

- The ids are sorted first, so the same data and seed always give the same split, on any computer.
- The seed search guarantees at least 2 boxes of every class in validation and test (or 1 if 2 is impossible). The chosen seed is saved in `split.json`.
- If `split.json` already exists in Drive, it is loaded instead of re-made. Put the shared file in your Drive folder *before* running this step.

Check: 114 / 24 / 25 images, and every class appears in all three lines of the printout.

Step 5 — Dataset, augmentation and the three models

Everyone reads this code carefully; it's the core of the viva

```

class TrashDataset(torch.utils.data.Dataset):
    def __init__(self, items, transforms=None):
        self.items, self.transforms = items, transforms

    def __len__(self):
        return len(self.items)

    def __getitem__(self, i):
        d = self.items[i]
        # uint8 image tensor, 3 x H x W
        img = tv_tensors.Image(TF.pil_to_tensor(Image.open(d["image"])).convert("RGB"))
        H, W = img.shape[-2:]
        boxes = torch.tensor(d["boxes"], dtype=torch.float32).reshape(-1, 4)
        target = {
            "boxes": tv_tensors.BoundingBoxes(boxes, format="XYXY", canvas_size=(H, W)),
            "labels": torch.tensor([CLASS_TO_ID[n] for n in d["names"]], dtype=torch.int64),
        }
        if self.transforms is not None:
            img, target = self.transforms(img, target)
        return img, target

train_tf = T.Compose([
    T.RandomPhotometricDistort(p=0.5),           # brightness, contrast, saturation, hue
    T.RandomZoomOut(fill={tv_tensors.Image: (124, 116, 104), "others": 0},
                     side_range=(1.0, 1.5), p=0.3), # zoom out: objects look further away
    T.RandomApply([T.RandomIoUCrop()], p=0.3),   # zoom in, keeping most of each object
    T.RandomHorizontalFlip(p=0.5),
    T.SanitizeBoundingBoxes(min_size=2),         # remove boxes the crop cut away
    T.ToDtype(torch.float32, scale=True),        # 0-255 -> 0-1
])
eval_tf = T.ToDtype(torch.float32, scale=True)   # no augmentation when measuring

```

- Wrapping the boxes as `tv_tensors.BoundingBoxes` tells `transforms.v2` that they are boxes, so every flip, zoom and crop moves them together with the pixels.
- `SanitizeBoundingBoxes` removes boxes (and their labels) that a crop cut away. `ToDtype(..., scale=True)` turns 0-255 pixels into 0-1 floats.
- `collate` returns lists rather than one stacked tensor, because detection images can have different sizes.

The three model builders. Each loads COCO weights and swaps the classification layer for 9 outputs:

```

SCORE_THRESH, NMS_THRESH, MAX_DETS = 0.01, 0.5, 100

def build_faster_rcnn(num_classes, pretrained=PRETRAINED):
    """Two-stage detector: region proposals first, then classify and refine each one."""
    weights = D.FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT if pretrained else None
    model = D.fasterrcnn_resnet50_fpn_v2(weights=weights, weights_backbone=None,
                                         min_size=MIN_SIZE, max_size=MAX_SIZE)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    # new last layer: 9 outputs (8 classes + background)
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    model.roi_heads.score_thresh = SCORE_THRESH
    model.roi_heads.nms_thresh = NMS_THRESH
    model.roi_heads.detections_per_img = MAX_DETS
    return model

def build_retinanet(num_classes, pretrained=PRETRAINED):
    """One-stage detector with focal loss: class + box for every anchor on every FPN level."""
    weights = D.RetinaNet_ResNet50_FPN_V2_Weights.DEFAULT if pretrained else None
    model = D.retinanet_resnet50_fpn_v2(weights=weights, weights_backbone=None,
                                         min_size=MIN_SIZE, max_size=MAX_SIZE)
    head = model.head.classification_head
    # keep the head's pretrained conv layers; replace only the last one (91 COCO classes -> 9)
    head.cls_logits = torch.nn.Conv2d(head.cls_logits.in_channels, head.num_anchors * num_classes,
                                       kernel_size=3, padding=1)
    torch.nn.init.normal_(head.cls_logits.weight, std=0.01)
    # start every class score at about 1% (the "prior", see the guide, section 5.2)
    torch.nn.init.constant_(head.cls_logits.bias, -math.log((1 - 0.01) / 0.01))
    head.num_classes = num_classes
    model.score_thresh, model.nms_thresh = SCORE_THRESH, NMS_THRESH
    model.detections_per_img = MAX_DETS
    return model

```

```
def build_ssd(num_classes, pretrained=PRETRAINED):
    """One-stage detector that predicts from six feature maps; fixed 300 x 300 input."""
    weights = D.SSD300_VGG16_Weights.DEFAULT if pretrained else None
    model = D.ssd300_vgg16(weights=weights, weights_backbone=None)
    in_channels = det_utils.retrieve_out_channels(model.backbone, (300, 300))
    num_anchors = model.anchor_generator.num_anchors_per_location()
    model.head.classification_head = SSDClassificationHead(in_channels, num_anchors, num_classes)
    model.score_thresh, model.nms_thresh = SCORE_THRESH, NMS_THRESH
    model.detections_per_img = MAX_DETS
    return model

MODELS = {"Faster R-CNN": build_faster_rcnn, "RetinaNet": build_retinanet, "SSD300": build_ssd}
```

- **Faster R-CNN:** `FastRCNNPredictor` is the last layer of stage 2 (class scores + box offsets per class). The RPN is class-agnostic, so it's kept as is.
- **RetinaNet:** only the final conv of the class subnet (`cls_logits`) is replaced, so the pretrained layers of the subnet are kept. Its bias starts at the 1% prior (section 5.2), and `num_classes` is updated because the head uses it to reshape its output.
- **SSD300:** the class predictors are one conv per feature map, so the whole classification head is rebuilt with the right input channels and anchors per map.
- The last lines of each builder give all three models the same post-processing: score ≥ 0.01 , NMS 0.5, at most 100 boxes.

Check: the augmentation preview shows four different versions of one photo, with boxes still on the objects.

Step 6 — Train

GPU time: about 20 + 20 + 5 minutes · model owners

```
def train_model(name, build, train_items, val_items):
    random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
    model = build(NUM_CLASSES).to(DEVICE)
    params = [p for p in model.parameters() if p.requires_grad]
    opt = torch.optim.AdamW(params, lr=LR, weight_decay=WEIGHT_DECAY)
    loader = make_loader(train_items, train_tf, shuffle=True, batch_size=BATCH_SIZE)
    total, warm = EPOCHS * len(loader), WARMUP_EPOCHS * len(loader)
    def lr_factor(s):
        # warm up for one epoch, then cosine decay to 0
        if s < warm:
            return (s + 1) / warm
        return 0.5 * (1 + math.cos(math.pi * (s - warm) / max(1, total - warm)))
    sched = torch.optim.lr_scheduler.LambdaLR(opt, lr_factor)
    scaler = torch.amp.GradScaler("cuda", enabled=USE_AMP)
    ckpt = CKPT_DIR / f"{slug(name)}.pt"
    history, best = [], -1.0
    for epoch in range(1, EPOCHS + 1):
        model.train()
        t0, sums = time.time(), defaultdict(float)
        for imgs, tgts in loader:
            imgs = [im.to(DEVICE) for im in imgs]
            tgts = [{"boxes": t["boxes"].to(DEVICE), "labels": t["labels"].to(DEVICE)}
                    for t in tgts]
            with torch.autocast(DEVICE.type, enabled=USE_AMP):
                losses = model(imgs, tgts) # in train mode the model returns a dict of losses
                loss = sum(losses.values())
            if not torch.isfinite(loss):
                raise RuntimeError(f"{name}: loss became {loss.item()}. "
                                   "Set USE_AMP = False or lower LR.")
            opt.zero_grad(set_to_none=True)
            scaler.scale(loss).backward()
            scaler.unscale_(opt)
            torch.nn.utils.clip_grad_norm_(params, max_norm=10.0)
            scaler.step(opt)
            scaler.update()
            sched.step()
            for k, v in losses.items():
                sums[k] += v.item() / len(loader)
        val = coco_map(*predict(model, val_items))
        row = {"epoch": epoch, "loss": sum(sums.values()), **sums, "val_map": val["map"],
              "val_map50": val["map_50"], "seconds": round(time.time() - t0, 1)}
        history.append(row)
```

```

note = ""
if val["map"] > best:
    # keep the best epoch, judged on validation only
    best, note = val["map"], " <- best so far, saved"
    torch.save(model.state_dict(), ckpt)
print(f"[{name}] epoch {epoch:>2}/{EPOCHS} loss {row['loss']:.3f} "
      f"val mAP {val['map']:.3f} mAP50 {val['map_50']:.3f} ({row['seconds']}s){note}")
(RESULTS_DIR / f"history_{slug(name)}.json").write_text(json.dumps(history, indent=1))
return history

```

- `params` contains only the trainable weights; the frozen early backbone layers are skipped.
- `lr_factor` is the schedule: a linear ramp for the first epoch, then half a cosine down to 0.
- In training mode, `model(imgs, tgts)` returns a **dict of losses**, whose names are explained in chapter 5.
- `GradScaler`, `unscale_` and `clip_grad_norm_` implement mixed precision with gradient clipping (section 3.4).
- After every epoch, validation mAP decides whether to save. Only the best epoch's weights end up in `checkpoints/<model>.pt`.

```

TRAIN_THESE = list(MODELS)          # e.g. ["RetinaNet"] if you are only training one model

def finished(name):
    f = RESULTS_DIR / f"history_{slug(name)}.json"
    complete = f.exists() and len(json.loads(f.read_text())) == EPOCHS
    return complete and (CKPT_DIR / f"{slug(name)}.pt").exists()

for name in TRAIN_THESE:
    if finished(name):
        print(f"{name}: already trained (delete its checkpoint to retrain)")
        continue
    train_model(name, MODELS[name], train_data, val_data)

```

Check: the printed validation mAP rises over the first epochs. `training_curves.png` shows the healthy pattern from section 7.3. Note the best epoch of each model for the report.

Step 7 — Results on the test set

5 minutes · evaluation owner (needs all three checkpoints)

First, the matching rule from section 8.1 in code:

```

def match_image(pred, gt, iou_thr=0.5):
    """Greedy matching, highest score first, same class only (the PASCAL VOC / COCO rule)."""
    n_p, n_g = len(pred["boxes"]), len(gt["boxes"])
    ious = box_iou(pred["boxes"], gt["boxes"]) if n_p and n_g else torch.zeros(n_p, n_g)
    taken = torch.zeros(n_g, dtype=torch.bool)
    is_tp = torch.zeros(n_p, dtype=torch.bool)
    matched = torch.full((n_p,), -1, dtype=torch.long)
    for i in range(n_p):
        # predictions are sorted by score: best first
        cand = ious[i].clone()
        # only boxes of the same class that no better prediction has claimed yet
        cand[(gt["labels"] != pred["labels"][i]) | taken] = -1
        if n_g and cand.max() >= iou_thr:
            j = int(cand.argmax())
            taken[j], is_tp[i], matched[i] = True, True, j
    return is_tp, matched, ious

```

```

def prf_at(rows, thr):
    """TP, FP, FN, precision, recall and F1 when we keep predictions with score >= thr."""
    counts = {c: [0, 0, 0] for c in range(1, NUM_CLASSES)} # class id -> [TP, FP, FN]
    for r in rows:
        keep = r["pred"]["scores"] >= thr
        for lab, ok in zip(r["pred"]["labels"][keep].tolist(), r["is_tp"][keep].tolist()):
            counts[lab][0 if ok else 1] += 1
        found = set(r["matched"][keep & r["is_tp"]].tolist())
        for j, lab in enumerate(r["gt"]["labels"].tolist()):
            if j not in found:
                counts[lab][2] += 1

```

```
def pr(tp, fp, fn):
    p = tp / (tp + fp) if tp + fp else 0.0
    rc = tp / (tp + fn) if tp + fn else 0.0
    return p, rc, (2 * p * rc / (p + rc) if p + rc else 0.0)
TP, FP, FN = (sum(v[i] for v in counts.values()) for i in range(3))
P, R, F1 = pr(TP, FP, FN)
fields = ("tp", "fp", "fn", "precision", "recall", "f1")
per_class = {CLASSES[c - 1]: dict(zip(fields, (*v, *pr(*v)))) for c, v in counts.items()}
return {"tp": TP, "fp": FP, "fn": FN, "precision": P, "recall": R, "f1": F1,
        "per_class": per_class}
```

```
def best_threshold(rows):
    grid = np.round(np.arange(0.05, 0.96, 0.05), 2)
    f1 = [prf_at(rows, t)["f1"] for t in grid]
    return float(grid[int(np.argmax(f1))])
```

Then, for every model: load the best checkpoint, choose the threshold on validation, and measure on test:

```
results = {}
for name in MODELS:
    if not (CKPT_DIR / f"{slug(name)}.pt").exists():
        print(f"{name}: no checkpoint yet, skipped")
        continue
    model = load_trained(name)
    val_rows = match_all(*predict(model, val_data))
    thr = best_threshold(val_rows) # chosen on validation, never on test
    test_preds, test_gts = predict(model, test_data)
    rows = match_all(test_preds, test_gts)
    m = {"threshold": thr, **coco_map(test_preds, test_gts, per_class=True), **prf_at(rows, thr)}
    rec, prec, _ = pr_curve(rows)
    m["ap50_pooled"] = average_precision(rec, prec)
    m["latency_ms"] = time_model(model, test_data)
    m["params_M"] = sum(p.numel() for p in model.parameters()) / 1e6
    m["size_MB"] = (CKPT_DIR / f"{slug(name)}.pt").stat().st_size / 1e6
    results[name] = {"metrics": m, "rows": rows}
    del model
    if DEVICE.type == "cuda":
        torch.cuda.empty_cache()
```

- `load_trained` rebuilds the architecture *without* downloading COCO weights, then loads our checkpoint.
- `best_threshold(val_rows)` uses **validation** only. The test set is predicted once, after the threshold is fixed.
- `coco_map` (torchmetrics) gives mAP@0.5:0.95, @0.5, @0.75, by size, AR@100 and AP per class. `prf_at` gives TP, FP, FN, P, R and F1 at the threshold. `time_model` gives the median latency.
- Outputs: `summary.csv`, `per_class.csv`, `metrics.json`, and the charts `comparison.png`, `per_class_ap.png` and `pr_curves.png`.

Step 8 — Error analysis

10 minutes · evaluation owner, then everyone discusses

```
def explain_errors(r, thr, iou_thr=0.5, near=0.1):
    p, g, ious = r["pred"], r["gt"], r["ious"]
    keep = p["scores"] >= thr
    same = p["labels"][:, None] == g["labels"][None, :] # [pred, gt]: same class?
    fps, fns = [], []
    for i in torch.where(keep & ~r["is_tp"])[0].tolist():
        iou_same = float((ious[i] * same[i]).max()) if len(g["labels"]) else 0.0
        iou_other = float((ious[i] * ~same[i]).max()) if len(g["labels"]) else 0.0
        if iou_same >= iou_thr:
            kind = "duplicate"
        elif iou_other >= iou_thr:
            kind = "wrong class"
        elif max(iou_same, iou_other) >= near:
            kind = "poor box"
        else:
            kind = "background"
        fps.append((i, kind))
    found = set(r["matched"][keep & r["is_tp"]].tolist())
```

```

for j in range(len(g["labels"])):
    if j in found:
        continue
    col = ious[:, j] if len(p["labels"]) else torch.zeros(0)
    s = same[:, j] if len(p["labels"]) else torch.zeros(0, dtype=torch.bool)
    if ((col >= iou_thr) & s & keep).any():
        kind = "merged with neighbour"
    elif ((col >= iou_thr) & s & ~keep).any():
        kind = "low confidence"
    elif ((col >= iou_thr) & ~s & keep).any():
        kind = "wrong class"
    elif ((col >= near) & keep).any():
        kind = "poor box"
    else:
        kind = "not detected"
    fns.append((j, kind))
return fps, fns

```

- `same` is a table saying, for each prediction-object pair, whether the classes agree. The rules then follow the order in section 9.1.
- The next cell counts the reasons per model into `errors.csv` and computes recall by object size. It then draws `error_breakdown.png`, `confusion_matrices.png` and the example panels `examples_<id>.png`.

Check: the TP count plus the "FN" rows equals the number of test objects for every model. Open the example panels and check that the colours make sense.

Step 9 — Unseen photos, and every laptop

30 minutes · everyone

Unseen photos. Put 5–10 of your own phone photos of waste into `MyDrive/trash_detection/unseen/` and run the cell. All three models draw their boxes side by side, and the figures are saved as `results/unseen_*.png`. There are no labels, so describe what you see: what was found, what was missed, what was hallucinated, and how it compares with the test photos. Different light, camera and backgrounds make this a genuine generalisation test.

Export. The last two cells write `model_info.json` (class names, thresholds, input size) and `predict.py` into the Drive folder. To put the project on a Windows laptop:

1. In Google Drive, open `trash_detection`, select everything except the large `trash_dataset.zip`, right-click → **Download** and unzip. It's about 450 MB, mostly the three checkpoints.
2. Install Python 3.11 or 3.12 from python.org (tick "Add python.exe to PATH"), then open a terminal in the unzipped folder:

```

pip install torch torchvision pillow
python predict.py --model ssd300 --images unseen --out predictions
python predict.py --model faster_rcnn --images unseen --out predictions
python predict.py --model retinanet --images unseen --out predictions

```

Annotated copies go to `predictions/` and the detections are printed. It runs on the CPU, at a few seconds per image for Faster R-CNN and RetinaNet and less for SSD300. Once every member has done this and has the report PDF in the same folder, the "project and report on every laptop" rule is covered.

11 Writing the report

The examiner reads the report to answer one question: *did this team run a sound comparison and understand what they found?* Every claim should rest on a number from your tables or a figure from `results/`. Aim for 12–18 pages including figures.

11.1 Structure

Section	What goes in	Source
Title page	Title, team members, course, date	
Abstract	5–6 sentences: problem, data, three models, headline result, recommendation	Write it last
1 Introduction	Why automatic waste detection matters; the objectives from the brief; what you did	Case study text
2 Dataset	Source and licence; 163 photos, 8 classes; class table; object sizes; cleaning (how many photos needed rotation or rescaling); the split (114 / 24 / 25, seed) and why you made your own	Step 2–4 printouts, <code>eda.png</code> , <code>label_check.png</code>
3 Methods	3.1 The three models (one paragraph each + the table from 5.4). 3.2 Training setup (recipe table + a fairness statement). 3.3 Evaluation protocol (matching rule, metrics, threshold on validation, speed measurement).	Chapters 5, 7, 8
4 Results	Overall table; per-class table; training curves; PR curves; speed vs accuracy; verdicts on H1–H4	<code>summary.csv</code> , <code>per_class.csv</code> , figures
5 Error analysis	Error table; error breakdown chart; confusion matrices; recall by object size; 3 example photos + unseen photos, each with a one-line caption	<code>errors.csv</code> , figures, Step 8 printout
6 Discussion	4–6 observations, each evidence → cause → implication (section 9.5)	Your analysis
7 Conclusion	Which model for which situation, and why, in numbers	
8 Limitations & future work	Honest limits and concrete next steps (lists below)	
References	Papers and tools you used	Appendix C
Appendix	Full settings; software versions (torch, torchvision, GPU); full per-class table	Step 0 printout

11.2 Tables to fill in

Table R1 · Overall results on the test set (25 photos)

Model	mAP@0.5:0.95	mAP@0.5	Precision	Recall	F1	Threshold	ms / image	Params (M)	Size (MB)
Faster R-CNN									
RetinaNet									
SSD300									

Table R2 · Per class: AP@0.5, with precision / recall at each model's threshold

Class	Test objects	Faster R-CNN	RetinaNet	SSD300
Bottle				

Class	Test objects	Faster R-CNN	RetinaNet	SSD300
Bottle Cap				
...	(all 8)			

Table E1 · Correct, false and missed detections on the test set

Outcome	Faster R-CNN	RetinaNet	SSD300
Correct (TP)			
False: duplicate / wrong class / poor box / background			
Missed: merged / low confidence / wrong class / poor box / not detected			
Recall on small / medium / large objects			

11.3 Writing the conclusion

Don't just name "the best model". Recommend one *for a situation*, and back it with numbers:

- **If accuracy matters most** (auditing waste streams, offline analysis): the model with the highest mAP@0.5:0.95, especially if it also leads on the most common classes.
- **If speed matters most** (a conveyor belt, cheap hardware): the fastest model whose recall on the important classes is still acceptable. Say what "acceptable" means, for example "above 0.8 recall on bottles and cans".
- **If small litter matters** (butts, caps): look at recall on small objects, not overall mAP.

End with what would most improve the results, taken from your error analysis. For example: "most misses were small objects, so higher-resolution input or more small-object examples would help most".

11.4 Limitations and future work

Limitations to state

- Small dataset (163 photos) and a small test set (25 photos)
- One training run per model, so no measure of run-to-run variation
- Shared settings, not tuned per model
- Photos from one source and environment (street trash, one collector)
- Labels by a single annotator; box tightness varies
- Speed measured on one GPU (Colab T4)

Future work to propose

- Collect and label more photos, especially rare and small classes
- k-fold cross-validation or bootstrap confidence intervals
- Per-model learning-rate search with an equal budget
- Higher resolution or image tiling for small objects
- Soft-NMS for touching objects
- Anchor-free (FCOS) and transformer detectors
- Test on real conveyor-belt images; compress the model for edge devices

11.5 Style tips

- Past tense for what you did ("we trained"); present tense for what the results show ("SSD300 misses more small objects").
- Define each metric once, in Methods. Use 3 decimals for mAP, P, R and F1 and whole numbers for ms, throughout.
- Every figure caption says what to *notice*. Turn printouts into tables; never paste raw notebook output.
- Where a result contradicts a hypothesis, say so and give a reason. It reads as understanding, not failure.

12 Viva preparation

"Every team member should understand the complete project and be able to explain any part of it." These 50 questions cover the whole project. Cover the answers and practise out loud in pairs. Answers marked [your results] depend on your numbers, so prepare them once your tables are final.

12.1 The problem and the data

1. Is this classification or detection? Why?

Detection. A photo can contain several items of different classes, and we need each item's class *and* location. Classification gives one label per photo.

2. Describe the dataset.

163 photos of street-collected trash with bounding boxes in 8 classes: Bottle, Bottle Cap, Can, Cardboard, Cigarette Box, Cigarette Butt, Cup, Plastic. Labels come in VOC, COCO and YOLO formats, plus resized and augmented copies. [your class counts and share of small objects]

3. Why didn't you use the 4,923 augmented images?

They are transformed copies of the same 163 photos. If a copy of a test photo is in training, the test score is inflated, which is data leakage. We split the originals and augment only the training photos, on the fly, with fresh variations every epoch.

4. How did you split the data, and why like that?

By photo, $70 / 15 / 15 = 114 / 24 / 25$, with a seed chosen so every class has at least two boxes in validation and in test. We used 15% for test rather than 10% to get a steadier estimate. The split is saved in split.json, so everyone used the same one.

5. What is EXIF orientation, and why did you handle it?

Phones store a rotation flag instead of rotating the pixels. If the boxes were drawn on the rotated view but we loaded the raw pixels, every box would be misplaced, silently. We compare each XML's recorded size with both versions of the image and use the matching one, then check by drawing the boxes.

6. Why Pascal VOC labels?

VOC stores xmin, ymin, xmax, ymax in pixels, exactly what torchvision detection models expect, so no conversion is needed.

7. Convert the VOC box (160, 120, 320, 360) in a 640×480 image to YOLO.

$cx = 240 / 640 = 0.375$, $cy = 240 / 480 = 0.5$, $w = 160 / 640 = 0.25$, $h = 240 / 480 = 0.5$.

8. Which classes are hardest, and why?

[your results] Usually the small ones (Cigarette Butt, Bottle Cap), which are only a few pixels, and the look-alikes (Cup / Bottle / Plastic, Cigarette Box / Cardboard).

12.2 The models

9. Why these three models?

They are three different designs: two-stage (Faster R-CNN), one-stage with focal loss (RetinaNet), and one-stage multi-scale and light (SSD300). They represent the classic trade-offs between accuracy and speed and between ways of handling background. All come from torchvision, so one pipeline trains all three fairly. YOLO wasn't allowed, and transformer detectors need far more data and training.

10. One-stage vs two-stage?

Two-stage proposes candidate regions, then classifies and refines each one: more accurate, slower. One-stage predicts class and box for every anchor in one pass: faster, but it has to cope with a huge majority of background anchors.

11. What is an anchor?

A reference box of a fixed size and shape at a grid position. For each anchor the model predicts whether an object is there, its class, and offsets to fit it.

12. What does the RPN do?

It scores 3 anchors per position on every FPN level as object or background and regresses them. After NMS at 0.7 it keeps about 1,000 (test) to 2,000 (training) proposals for stage 2.

13. What is RoIAlign?

It samples a fixed 7×7 grid of features for each proposal from the shared feature map, using bilinear interpolation without rounding. It's cheap because the backbone runs only once, and precise because nothing is rounded.

14. Explain focal loss.

$FL = -\alpha_t(1 - p_t)^\gamma \log p_t$. The $(1 - p_t)^\gamma$ factor shrinks the loss of well-classified examples: at $p_t = 0.9$ it is 100 times smaller with $\gamma = 2$. The roughly 160,000 easy background anchors then don't drown the few objects. α balances positives and negatives.

15. Why does RetinaNet's last layer start at a 1% prior?

So that every anchor initially predicts about a 1% chance of an object. Starting at 50% would give a huge loss from the many background anchors and an unstable start.

16. How does SSD handle the background imbalance?

Hard negative mining: only the highest-loss background boxes count, 3 for every positive.

17. Why does SSD300 struggle with small objects?

The photo is squashed to 300×300 , so a 40-px cigarette butt in a 1,024-px photo becomes about 12 px. Its finest map is 38×38 (about 8 px per cell), and it has no FPN to bring deeper features to that map.

18. What is an FPN?

A top-down path with lateral connections that adds deep, meaningful features to the high-resolution maps. Every pyramid level is then both sharp and semantic, and small objects are detected on the fine levels.

19. What is NMS, and which threshold did you use?

For each class: keep the highest-scoring box, delete boxes overlapping it by IoU above 0.5, and repeat. We used 0.5 for all three models, for fairness.

20. What did you change in each model for 8 classes?

Only the final classification layer, now 9 outputs (8 + background): the FastRCNNPredictor in Faster R-CNN; the `cls_logits conv` in RetinaNet, with its bias set to the 1% prior; the classification head in SSD300. Everything else keeps its COCO weights and is fine-tuned.

21. Softmax or sigmoid?

Faster R-CNN and SSD300 use a softmax over 9 classes, including background, so exactly one class wins. RetinaNet uses an independent sigmoid per class: background simply means every class score is low.

22. What does "v2" mean?

torchvision's improved versions: better heads (BatchNorm in Faster R-CNN, GroupNorm in RetinaNet), a GloU box loss for RetinaNet and longer training. On COCO, 46.7 vs 37.0 mAP for Faster R-CNN and 41.5 vs 36.4 for RetinaNet.

12.3 Training

23. What is transfer learning, and why use it here?

Starting from weights learned on a big dataset (COCO) and fine-tuning on ours. 114 photos can't teach visual features from scratch, and COCO even contains bottles and cups.

24. Which optimizer and learning rate, and why the same for all three?

AdamW, learning rate $1e-4$, weight decay $1e-4$. Its per-weight step sizes make it robust to the exact learning rate, so one standard fine-tuning setting is fair to all three models.

25. Why warm-up?

The new last layer is random at the start. Full-size steps from the first iteration would push its large random gradients into the pretrained layers. Ramping up over one epoch avoids that shock.

26. How did you choose the number of epochs?

We trained for 30 and kept the epoch with the best validation mAP@0.5:0.95, which is early stopping. [your best epochs, and whether the curves had levelled off]

27. How do you know you didn't overfit?

Validation mAP was measured every epoch on photos never used for training, and we kept the best epoch. Augmentation, weight decay and pretrained weights also help. [describe your curves]

28. What is mixed precision?

Doing most of the maths in 16-bit floats, which is faster on the GPU's tensor cores, with a gradient scaler so small gradients don't round to zero. Accuracy is effectively unchanged.

29. Why not compare loss values between models?

Each model defines its loss differently: four terms for Faster R-CNN, focal + GloU for RetinaNet, cross-entropy + smooth L1 for SSD300. Only trends within one model mean anything, so we compare mAP.

30. What does a torchvision detector return in training vs evaluation mode?

In training mode (given targets), a dict of losses. In evaluation mode, a list with one dict per image of boxes, labels and scores.

12.4 Metrics

31. Define IoU and compute an example.

Overlap area $\div$ union area. For ground truth (100, 100, 300, 300) and prediction (150, 150, 350, 350): 22,500 / 57,500 = 0.39, so it's not a match at 0.5.

32. When is a detection a true positive?

Same class, IoU ≥ 0.5 with a ground-truth box that isn't already matched, with predictions matched in order of confidence.

33. Why is there no accuracy?

Detection has no countable true negatives: there are countless background regions, so accuracy can't be defined. We use precision and recall.

34. Precision vs recall for a waste sorter?

Precision is the share of our boxes that are right; recall is the share of objects we find. Low recall means recyclables slip through; low precision means wrong picks and contamination.

35. What is AP, and what is mAP?

AP is the area under the interpolated precision-recall curve for one class, sweeping the confidence threshold. mAP is the mean of AP over the classes.

36. mAP@0.5 vs mAP@0.5:0.95?

The first counts IoU ≥ 0.5 as correct. The second averages mAP over IoU 0.50 to 0.95, so it also rewards tight boxes, and it's always lower.

37. How did you choose the confidence threshold?

For each model, the value with the best F1 on the validation set. Precision, recall and F1 on the test set are reported at that threshold. Choosing it on the test set would be tuning to the test set.

38. How did you measure speed?

The median time over 20 test photos at batch 1 on the T4 with mixed precision, after 3 warm-up runs, with `torch.cuda.synchronize()` around each timing. It includes resizing and NMS but not reading from disk.

39. How reliable are results on 25 test photos?

Moderately. One object can shift a class's AP a lot, so we rely on clear gaps and consistent patterns. Future work: bootstrap confidence intervals or k-fold cross-validation.

40. Why use torchmetrics *and* your own matching code?

torchmetrics gives the standard COCO mAP numbers. Our own matching gives TP / FP / FN at the chosen threshold and a reason for each error. We checked that they agree: our AP@0.5 matched the COCO-style value to within 0.001.

12.5 Results and analysis

41. Which model is best?

[your results] "Most accurate: __ (mAP@0.5:0.95 = __). Fastest: __ (__ ms per image). For a real-time sorting line I would recommend __, because __."

42. What were the most common errors?

[from errors.csv] Name the top one or two error types per model, with the cause (table in section 9.2).

43. Which classes were confused with each other?

[from the confusion matrices], and why: similar shape, material or colour.

44. Did your hypotheses hold?

[a verdict on H1–H4, with the numbers]

45. How did the models do on your own photos?

[your observations] Differences come from domain shift: other light, camera, background and object styles.

46. What would you do with two more weeks?

Label more photos, especially rare and small classes; higher resolution or tiling for small objects; per-model tuning; k-fold cross-validation; try FCOS or a transformer detector; compress the best model for a device.

12.6 Practical

47. Walk me through your pipeline.

Download → clean the labels (rotation, resize) → check them by eye → split 70 / 15 / 15 → dataset with augmentation → train three models with one recipe, keeping the best validation epoch → choose thresholds on validation → test metrics → error analysis → unseen photos → report.

48. Show it running.

```
python predict.py --model ssd300 --images unseen (the fastest on a laptop CPU), then the same with faster_rcnn.
```

49. Why is `num_classes` 9 and not 8?

torchvision reserves label 0 for background, so our classes are 1–8 and the output layer needs 9 slots.

50. What does `SanitizeBoundingBoxes` do?

After a random crop or zoom, it removes boxes that became empty or tiny, along with their labels, so no invalid box reaches the model.

13 Troubleshooting

Symptom	Likely cause	Fix
"CUDA out of memory"	Batch too large for the T4	<code>BATCH_SIZE = 2</code> , then Runtime → Restart and run again. If it still fails, set <code>MIN_SIZE</code> , <code>MAX_SIZE = 640</code> , <code>1066</code> for <i>all</i> models.
Output says <code>device: cpu</code>	GPU runtime not selected, or quota used up	Runtime → Change runtime type → T4 GPU. If none is available, try later or from a teammate's account.
Download fails, or "not a zip file"	Kaggle refused the direct link	Download in the browser and upload to <code>MyDrive/trash_detection/trash_dataset.zip</code>
0 images found, or 4,000+	<code>ORIG_DIR</code> points at the wrong folder	Look at the tree printout again and choose the original-data folder
9 classes printed	A spelling variant in the labels	Add a replacement in <code>clean_name</code> , e.g. <code>.replace("Cigarettes", "Cigarette")</code>
Boxes rotated or shifted in <code>label_check.png</code>	An orientation or scale case the rules missed	Compare one XML's <code><size></code> with the image's real size. If the dataset's resized folder has matching labels, try pointing <code>ORIG_DIR</code> there. Don't train until the boxes fit.
"All bounding boxes should have positive height and width"	A box with <code>xmax ≤ xmin</code>	The clean copy drops these; if it still appears, find the XML with swapped coordinates
Loss becomes <code>nan</code>	16-bit overflow, or learning rate too high	<code>USE_AMP = False</code> and/or <code>LR = 5e-5</code> , then retrain that model
Validation mAP ≈ 0 after 5+ epochs	Misaligned labels, wrong class mapping, or <code>PRETRAINED = False</code>	Re-check <code>label_check.png</code> , confirm <code>PRETRAINED = True</code> , then run the sanity check below
<code>KeyError</code> when the split loads	<code>split.json</code> was made from a different folder	Delete it, re-make it, and re-share the new file with everyone
Colab disconnected during training	Idle or time limits	Re-run from the top. Finished models are skipped; the interrupted one restarts.
Numbers change slightly between runs	GPU non-determinism	Normal. Report one run with its seed and mention it under limitations.
<code>predict.py</code> : "No module named torch"	Not installed on the laptop	<code>pip install torch torchvision pillow</code>
<code>predict.py</code> : file not found	Run from another folder	<code>cd</code> into the project folder; check <code>checkpoints/</code> and <code>model_info.json</code> are there
torchmetrics complains about pycocotools	Package missing	<code>!pip install pycocotools</code> , then restart the runtime

13.1 The two-minute sanity check

If you suspect a bug in the pipeline, check whether a model can **memorise four photos**. A working pipeline learns them quickly, and the score should climb clearly (usually past 0.5 within these 50 steps). If it stays at 0.00, the boxes or labels reaching the model are wrong, so go back to Steps 2–3. Paste this into a new cell after Step 6's first cell. It saves under its own name, so it doesn't touch your real checkpoints.

```
# Sanity check: can Faster R-CNN memorise 4 training photos? (about 2 minutes on a T4)
EPOCHS_SAVED, EPOCHS = EPOCHS, 10
hist = train_model("Sanity check", build_faster_rcnn, train_data[:4] * 5, train_data[:4])
EPOCHS = EPOCHS_SAVED
print("best mAP@0.5 on the 4 photos it trained on:", round(max(r["val_map50"] for r in hist), 2))
# afterwards, delete checkpoints/sanity_check.pt and results/history_sanity_check.json
```

A Formula sheet

IoU	$\frac{\text{area}(A \cap B)}{\text{area}(A) + \text{area}(B) - \text{area}(A \cap B)}$
VOC → YOLO	$cx = (x_{\min} + x_{\max}) / 2W, cy = (y_{\min} + y_{\max}) / 2H, w = (x_{\max} - x_{\min}) / W, h = (y_{\max} - y_{\min}) / H$
VOC → COCO	$[x_{\min}, y_{\min}, x_{\max} - x_{\min}, y_{\max} - y_{\min}]$
Box encoding	$t_x = (x - x_a) / w_a, t_y = (y - y_a) / h_a, t_w = \log(w / w_a), t_h = \log(h / h_a)$
Precision, recall	$P = TP / (TP + FP), R = TP / (TP + FN)$
F1	$2PR / (P + R)$
AP (all-point)	$\sum_k (r_k - r_{k-1}) \cdot p_{\text{interp}}(r_k), \text{ where } p_{\text{interp}}(r) = \text{max precision at recall } \geq r$
mAP	Mean of AP over the 8 classes
mAP@0.5:0.95	Mean of mAP at IoU = 0.50, 0.55, ..., 0.95 (10 values)
Focal loss	$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log p_t, \text{ with } \gamma = 2, \alpha = 0.25$
RetinaNet prior bias	$b = -\log((1 - \pi) / \pi) = -4.6 \text{ for } \pi = 0.01$
SSD300 default boxes	$38^2 \cdot 4 + 19^2 \cdot 6 + 10^2 \cdot 6 + 5^2 \cdot 6 + 3^2 \cdot 4 + 1^2 \cdot 4 = 8,732$
Learning-rate schedule	$lr(s) = lr_0 (s + 1) / W \text{ for } s < W; \text{ otherwise } lr_0 \cdot \frac{1}{2} (1 + \cos(\pi (s - W) / (S - W))), \text{ where } W = \text{warm-up steps and } S = \text{total steps}$
Iterations	114 images ÷ batch 4 = 29 per epoch; × 30 epochs ≈ 870 updates
Object size used here	$\sqrt{(\text{box area} \div \text{image area})}: \text{small} < 0.1 \leq \text{medium} < 0.3 \leq \text{large}$
COCO sizes	small < 32 ² px, medium 32 ² –96 ² , large > 96 ² (by box area)

B Glossary

Anchor / default box	A reference box of fixed size and shape at a grid position, which the model classifies and adjusts.
AP, mAP	Area under the precision–recall curve for one class; its mean over classes.
AR@100	Average recall with up to 100 detections per image, averaged over IoU 0.5–0.95.
Augmentation	Random changes to training images (flip, zoom, colour) that create new variations; boxes move with the pixels.
Backbone	The network that turns an image into feature maps (ResNet-50, VGG-16).
Background	Class 0 in torchvision: "no object here".
Batch, iteration, epoch	Images processed together; one weight update; one pass over all training images.
Checkpoint	A saved copy of the model's weights (.pt file).
COCO	A large dataset (80 object types) and the standard set of detection metrics named after it.
Confidence score	The model's certainty for a box, between 0 and 1.
Confusion matrix	A table of true class against predicted class; in detection it includes a background row and column.
Data leakage	Test information reaching training (e.g., augmented copies of test photos), which inflates scores.
EXIF orientation	A phone's "rotate when displaying" flag stored inside a JPEG.
Feature map	A grid of feature vectors produced by a layer; each cell describes a patch of the image.
Fine-tuning	Continuing to train pretrained weights on new data with a small learning rate.
Focal loss	Cross-entropy scaled by $(1 - p_t)^{\gamma}$ so easy examples count less (RetinaNet).
FPN	Feature Pyramid Network: combines deep and shallow maps so every level is sharp and meaningful.
GFLOPs	Billions of floating-point operations per image: a hardware-independent measure of compute.
GIoU	Generalised IoU: IoU minus a penalty for empty space in the smallest enclosing box; used as a box loss.
Ground truth	The human-labelled boxes and classes.
Hard negative mining	Keeping only the highest-loss background examples (SSD300: 3 per positive).
Head	The part of the network that outputs class scores and box offsets.
IoU	Intersection over union: $\text{overlap} \div \text{combined area of two boxes}$.
Latency, FPS	Time per image; images per second ($1000 \div \text{latency in ms}$).
Mixed precision (AMP)	Training with 16-bit floats where safe, for speed.
NMS	Non-maximum suppression: removes lower-scoring boxes that overlap a better box of the same class.
One-stage / two-stage	Predict everything in one pass / propose regions first, then classify them.
Overfitting	Memorising the training data: training loss falls while validation performance stalls or drops.
Pascal VOC	A classic detection dataset and its XML label format (xmin, ymin, xmax, ymax).
Precision, recall, F1	Share of boxes that are right; share of objects found; their harmonic mean.
Proposal	A candidate region produced by the RPN.
RoIAlign	Samples a fixed-size feature patch for each proposal, without rounding.
RPN	Region Proposal Network, stage 1 of Faster R-CNN.
Smooth L1	A box loss that is quadratic for small errors and linear for large ones.
Threshold (confidence)	The minimum score for a box to be kept when computing precision and recall.
TP, FP, FN	Correct detection; false detection; missed object.
Transfer learning	Reusing a model trained on one task or dataset as the starting point for another.
Validation set	Held-out photos used to make decisions (best epoch, threshold), kept separate from the test set.
Warm-up	Starting training with a very small learning rate that ramps up.

C References

1. Ren, S., He, K., Girshick, R., Sun, J. (2015). *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. NeurIPS. arXiv:1506.01497
2. Lin, T.-Y., Goyal, P., Girshick, R., He, K., Dollár, P. (2017). *Focal Loss for Dense Object Detection* (RetinaNet). ICCV. arXiv:1708.02002
3. Liu, W., et al. (2016). *SSD: Single Shot MultiBox Detector*. ECCV. arXiv:1512.02325
4. Lin, T.-Y., et al. (2017). *Feature Pyramid Networks for Object Detection*. CVPR. arXiv:1612.03144
5. Li, Y., Xie, S., Chen, X., Dollár, P., He, K., Girshick, R. (2021). *Benchmarking Detection Transfer Learning with Vision Transformers* (basis of Faster R-CNN v2). arXiv:2111.11429
6. Zhang, S., Chi, C., Yao, Y., Lei, Z., Li, S. Z. (2020). *Bridging the Gap Between Anchor-based and Anchor-free Detection via Adaptive Training Sample Selection* (reference for RetinaNet v2). CVPR. arXiv:1912.02424
7. Rezatofighi, H., et al. (2019). *Generalized Intersection over Union*. CVPR. arXiv:1902.09630
8. Bolya, D., Foley, S., Hays, J., Hoffman, J. (2020). *TIDE: A General Toolbox for Identifying Object Detection Errors*. ECCV. arXiv:2008.08115
9. Lin, T.-Y., et al. (2014). *Microsoft COCO: Common Objects in Context*. ECCV. arXiv:1405.0312
10. Everingham, M., et al. (2010). *The PASCAL Visual Object Classes (VOC) Challenge*. IJCV 88, 303–338.
11. He, K., Zhang, X., Ren, S., Sun, J. (2016). *Deep Residual Learning for Image Recognition* (ResNet). CVPR. arXiv:1512.03385
12. Simonyan, K., Zisserman, A. (2015). *Very Deep Convolutional Networks for Large-Scale Image Recognition* (VGG). ICLR. arXiv:1409.1556
13. Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization* (AdamW). ICLR. arXiv:1711.05101
14. Dataset: *Labeled Trash Dataset* (Pascal VOC, COCO and YOLO), Kaggle, user pandapowr. kaggle.com/datasets/pandapowr/labeled-trash-dataset-pascal-voc-coco-and-yolo
15. torchvision object-detection models and weights: pytorch.org/vision/stable/models.html
16. TorchVision Object Detection Finetuning Tutorial: pytorch.org/tutorials/intermediate/torchvision_tutorial.html
17. torchmetrics Mean Average Precision: lightning.ai/docs/torchmetrics/stable/detection/mean_average_precision.html